String Templates

Core idioms first, then problems grouped by pattern.
For sliding window string problems (#3, #76, #567, #424), see `sliding_window.md`.

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
242	Valid Anagram	Return true if `t` is an anagram of `s` (same characters, same counts).	Anagrams have identical letter counts, so add up `s` and subtract `t` in one 26-slot array — all zeros means they match.	O(n)	O(1)	Standard
383	Ransom Note	Can `ransomNote` be constructed from the letters in `magazine` (each letter used at most once)?	Stock the letter counts from the magazine, then draw down for each note letter — if any count goes negative the magazine is short.	O(n)	O(1)	Standard
387	First Unique Character in a String	Return the index of the first non-repeating character. Return -1 if none exists.	One pass to count every letter, a second pass to return the first whose count is exactly 1.	O(n)	O(1)	Standard
49	Group Anagrams	Group strings that are anagrams of each other.	Anagrams share identical letter frequencies, so the frequency vector encoded as a string is a unique group key — O(k) to build vs O(k log k) to sort the characters.	O(nk)	O(n)	Standard
451	Sort Characters By Frequency	Sort characters in descending order of frequency. Ties can go in any order.	Frequencies are bounded by string length, so bucket chars by their count and read buckets high-to-low — no O(n log n) comparison sort needed.			Standard
5	Longest Palindromic Substring	Return the longest palindromic substring.	Every palindrome has a center; try all 2n-1 centers (each char and each gap) and grow outward while characters mirror.	$O(n^2)$	O(1)	Standard
647	Palindromic Substrings	Count all palindromic substrings.	Same center-expansion, but each successful step outward is itself one more palindrome, so accumulate a count rather than a max.	$O(n^2)$	O(1)	Standard
8	String to Integer (atoi)	Parse a string to an integer, handling leading spaces, optional sign, overflow, and non-digit stop.	Process the string in strict phases — spaces, then sign, then digits accumulated as `num*10 + digit` — clamping to int bounds the moment overflow would occur.	O(n)	O(1)	Standard
14	Longest Common Prefix	Find the longest common prefix string among an array of strings.	Start with the first string as the candidate prefix and trim its tail until every other string starts with it.	O(n·k)	O(1)	Standard
443	String Compression	Compress the char array in-place. Each group `aaa` becomes `a3`. Single chars stay as-is. Return new length.	Two pointers — read scans each run while write lays down the compressed `char + count` behind	O(n)	O(1)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
			it; write never overtakes read, so it's safe in place.			
1047	Remove All Adjacent Duplicates in String	Repeatedly remove adjacent equal characters until no more exist.	A stack mirrors the partially-cleaned string — if the next char equals the top it's an adjacent pair, so pop instead of pushing.	O(n)	O(n)	Standard
6	ZigZag Conversion	Convert a string into a zigzag pattern across `numRows` rows and read it row by row.	Walk the string once, appending each char to its current row's builder while bouncing the row index down then up.	O(n)	O(n)	Standard
12	Integer to Roman	Convert an integer to its Roman numeral representation (values 1–3999).	Greedily subtract the largest possible value (including the 4/9 subtractive pairs) and append its symbol.	O(1)	O(1)	Standard
13	Roman to Integer	Convert a Roman numeral string to an integer.	Add each symbol's value, but subtract instead when a smaller value precedes a larger one (subtractive notation).	O(n)	O(1)	Standard
28	Implement strStr()	Find the first occurrence of `needle` in `haystack`. Return -1 if not found.	Slide a window of needle's length across haystack and compare; the first full match wins.	O(n·m)	O(1)	Standard
38	Count and Say	Generate the nth term of the "count and say" sequence: read digits of the previous term aloud.	Starting from "1", repeatedly scan runs of equal digits and emit count followed by the digit.	O(n·m)	O(m)	Standard
43	Multiply Strings	Multiply two non-negative integers given as strings. Return the product as a string.	Schoolbook multiplication: digit i of num1 times digit j of num2 lands in result positions i+j and i+j+1.	O(n·m)	O(n+m)	Standard
58	Length of Last Word	Return the length of the last word in a string (word = max sequence of non-space chars).	Scan from the right: skip trailing spaces, then count letters until the next space.	O(n)	O(1)	Standard
65	Valid Number	Validate whether a string is a valid decimal number (integers, fractions, exponents).	A single left-to-right scan tracking whether we have seen a digit, a dot, and an exponent enforces all the ordering rules.	O(n)	O(1)	Standard
68	Text Justification	Format words into lines of `maxWidth`, fully justified (equal spaces, last line left-aligned).	Greedily pack as many words as fit per line, then distribute leftover spaces evenly between gaps (extras to the left); the last line is left-justified.	O(n·maxWidth)	O(n)	Standard
125	Valid Palindrome	Check if a string is a palindrome, ignoring non-alphanumeric characters and case.	Two pointers move inward, skipping non-alphanumeric chars and comparing lowercased letters.	O(n)	O(1)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
151	Reverse Words in a String	Reverse the order of words in a string (split on spaces, handle multiple spaces).	Scan from the right, extract each word, and append them in reverse order with single spaces.	O(n)	O(n)	Standard
161	One Edit Distance	Determine if two strings are one edit distance apart (insert, delete, or replace one char).	Scan to the first mismatch; equal lengths force a replace, otherwise skip one char in the longer string and require the rest to match.	O(n)	O(n)	Standard
163	Missing Ranges	Given a sorted array, return missing ranges between `lower` and `upper` bounds.	Walk a running "next expected" value; whenever a number exceeds it, the gap before it is a missing range.	O(n)	O(1)	Standard
165	Compare Version Numbers	Compare two version number strings (dot-separated integers). Return -1, 0, or 1.	Parse both revisions position by position, treating missing components as 0, and compare numerically.	O(n)	O(n)	Standard
166	Fraction to Recurring Decimal	Convert a fraction numerator/denominator to a decimal string, noting repeating parts in parentheses.	Do long division; remember the position of each remainder so a repeat reveals the cycle to wrap in parentheses.	O(den)	O(den)	Standard
179	Largest Number	Arrange numbers so their concatenation forms the largest possible number.	Sort by a custom comparator that prefers the order producing a larger concatenation (a+b vs b+a).	O(n log n · k)	O(n)	Standard
186	Reverse Words in a String II	Reverse words in a character array in-place (words separated by spaces).	Reverse the whole array, then reverse each word back so the order flips but each word reads forward.	O(n)	O(1)	Standard
205	Isomorphic Strings	Check if two strings follow the same character-substitution pattern (isomorphic).	Maintain a bijective mapping both ways; any conflicting mapping breaks isomorphism.	O(n)	O(1)	Standard
214	Shortest Palindrome	Find the shortest palindrome by adding characters in front of a string.	Find the longest palindromic prefix via KMP failure on `s + '#' + reverse(s)`; reverse the leftover suffix and prepend it.	O(n)	O(n)	Standard
228	Summary Ranges	Given a sorted unique integer array, summarize consecutive runs as ranges.	Track the start of each consecutive run; when the run breaks, emit either a single number or a "start->end" range.	O(n)	O(1)	Standard
246	Strobogrammatic Number	Check if a number reads the same when rotated 180 degrees.	Two pointers move inward; each pair must be a valid strobogrammatic mapping (0-0, 1-1, 6-9, 8-8, 9-6).	O(n)	O(1)	Standard
249	Group Shifted Strings	Group strings that follow the same shift sequence (each	Encode each string by the gaps between consecutive chars (mod 26) so all	O(n·k)	O(n·k)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
		char shifted by the same amount).	members of a shift group share the same key.			
266	Palindrome Permutation	Check if a string can be rearranged into a palindrome (at most one odd-count char).	A palindrome allows at most one character with an odd count; track parity with a set.	O(n)	O(1)	Standard
271	Encode and Decode Strings	Encode a list of strings to a single string and decode it back.	Length-prefix each string ("len#str") so the decoder always knows exactly how many chars to read, regardless of content.	O(n)	O(n)	Standard
273	Integer to English Words	Convert a non-negative integer to its English words representation.	Process the number in groups of three digits, naming each group and appending the right scale word (Thousand, Million, Billion).	O(1)	O(1)	Standard
290	Word Pattern	Check if a string `s` follows a given pattern (bijective character-to-word mapping).	Maintain a two-way mapping between pattern chars and words; any conflict breaks the bijection.	O(n)	O(n)	Standard
299	Bulls and Cows	Count bulls (right digit, right place) and cows (right digit, wrong place) in a number guessing game.	Count exact-position matches as bulls; for the rest, use a digit-count array where positive entries (secret surplus) and negative entries (guess surplus) reveal cows.	O(n)	O(1)	Standard
344	Reverse String	Reverse a character array in-place.	Two pointers swap from both ends moving inward.	O(n)	O(1)	Standard
345	Reverse Vowels of a String	Reverse only the vowels of a string.	Two pointers advance until each lands on a vowel, then swap those vowels.	O(n)	O(n)	Standard
388	Longest Absolute File Path	Find the length of the longest absolute file path in a file system string.	Tab depth gives the nesting level; keep a stack of path lengths per level and, on hitting a file (has a dot), compute the full path length.	O(n)	O(n)	Standard
392	Is Subsequence	Check if string `s` is a subsequence of string `t`.	Walk `t` with one pointer; advance the `s` pointer each time chars match. If `s` is exhausted, it is a subsequence.	O(n)	O(1)	Standard
408	Valid Word Abbreviation	Check if an abbreviation matches a word (digits represent skipped characters).	Walk both with pointers; on a digit, parse the skip count (no leading zero) and jump the word pointer; otherwise chars must match.	O(n)	O(1)	Standard
409	Longest Palindrome	Find the length of the longest palindrome that can be built from the given letters.	Use every pair of each letter; if any letter has a leftover single, one of them can sit in the center.	O(n)	O(1)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
415	Add Strings	Add two non-negative integers given as strings. Return the sum as a string.	Add digit by digit from the right with a carry, exactly like grade-school addition.	O(max(n,m))	O(max(n,m))	Standard
422	Valid Word Square	Given a sequence of words, check whether they form a valid word square (kth row equals kth column).	For each cell (i,j), the char must equal the char at (j,i); guard against ragged rows by bounds-checking.	O(n·m)	O(1)	Standard
423	Reconstruct Original Digits from English	Given a scrambled string of digit-word letters, decode the original digits in order.	Some letters are unique to one digit (z→zero, w→two, u→four, x→six, g→eight); count those first, then peel off the remaining digits using letters that become unique after subtraction.	O(n)	O(1)	Standard
434	Number of Segments in a String	Count the number of segments (maximal runs of non-space chars) in a string.	A new segment starts at each non-space char whose predecessor is a space (or string start).	O(n)	O(1)	Standard
459	Repeated Substring Pattern	Check if a string can be built by repeating one of its substrings multiple times.	If `s` is a repeated pattern, it appears inside `(s+s)` with the first and last char removed.	O(n)	O(n)	Standard
468	Validate IP Address	Validate whether a string is a valid IPv4 or IPv6 address.	Dispatch on the separator: dots mean IPv4 (four 0–255 octets, no leading zeros), colons mean IPv6 (eight 1–4 hex groups).	O(n)	O(n)	Standard
500	Keyboard Row	Find all words that can be typed on a single row of a QWERTY keyboard.	Map each letter to its row index, then keep words whose letters all share one row.	O(n·k)	O(n)	Standard
520	Detect Capital	Check if a word is capitalized correctly (all caps, all lower, or first letter only).	Count uppercase letters: valid only if all are uppercase, none are, or exactly the first one is.	O(n)	O(1)	Standard
524	Longest Word in Dictionary through Deleting	Find the longest word in a dictionary that is a subsequence of string `s` (smallest lexicographically on ties).	Check each candidate with the subsequence two-pointer, keeping the best by length then lexicographic order.	O(n·k)	O(1)	Standard
541	Reverse String II	Reverse the first `k` characters of every `2k`-character block in a string.	Step through the array in `2k` jumps, reversing the first `k` chars of each block (or all that remain).	O(n)	O(n)	Standard
551	Student Attendance Record I	Check if a record is award-eligible (fewer than 2 absences total, no 3 consecutive lates).	Count total absences and track the current run of lates; fail on the second absence or third consecutive late.	O(n)	O(1)	Standard
556	Next Greater Element III	Find the next greater number by rearranging the digits of `n`. Return -1 if none or on overflow.	Standard next-permutation on digits: find the rightmost ascending pair, swap with the next larger digit to its right, reverse the suffix.	O(d)	O(d)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
557	Reverse Words in a String III	Reverse individual words in a string while preserving word order.	Split on spaces, reverse each word, and rejoin with single spaces.	O(n)	O(n)	Standard
592	Fraction Addition and Subtraction	Add or subtract a sequence of fractions, returning the result in lowest terms.	Parse each signed fraction, accumulate over a common denominator, then reduce by the GCD.	O(n)	O(1)	Standard
616	Add Bold Tag in String	Wrap every substring of `s` that matches a word in the dictionary with `<b>...</b>`, merging overlaps.	Mark every covered index in a boolean array, then emit bold tags around maximal marked runs.	O(n·m)	O(n)	Standard
657	Robot Return to Origin	Check if a robot following an instruction string (UDLR) returns to the origin.	Track net horizontal and vertical displacement; the robot returns only if both are zero.	O(n)	O(1)	Standard
670	Maximum Swap	Swap two digits at most once to get the maximum possible value.	Record the last index of each digit; scanning left to right, swap the first digit with the largest later digit that exceeds it.	O(d)	O(1)	Standard
680	Valid Palindrome II	Check if a string is a palindrome after deleting at most one character.	Two pointers inward; at the first mismatch, try skipping either the left or right char and check the remainder.	O(n)	O(1)	Standard
681	Next Closest Time	Find the next closest valid time using only the digits present in a given time string.	From the current minute, advance one minute at a time and return the first time whose digits are all in the allowed set.	O(1)	O(1)	Standard
686	Repeated String Match	Find the minimum number of times string `a` must repeat so `b` is a substring. Return -1 if impossible.	Repeat `a` until its length covers `b`, then check one extra copy to handle offset overlap.	O(n·m)	O(n)	Standard
709	To Lower Case	Convert a string to lowercase.	Uppercase ASCII letters differ from lowercase by 32, so shift any A–Z char.	O(n)	O(n)	Standard
726	Number of Atoms	Parse a chemical formula string and return atom counts in sorted order.	Recursive-descent style parse with a stack of count maps for parentheses; multiply a group's counts by the number following its closing paren.	O(n)	O(n)	Standard
748	Shortest Completing Word	Find the shortest word in a list that contains all letters of a license plate (case-insensitive, with multiplicity).	Build the plate's letter-count vector, then keep the shortest word whose own counts cover it.	O(n·k)	O(1)	Standard
788	Rotated Digits	Count numbers in `[1, n]` that become a different valid number when each digit is rotated 180 degrees.	A number is "good" if all digits are valid rotations and at least one digit (2,5,6,9) actually changes.	O(n·d)	O(1)	Standard
791	Custom Sort String	Sort string `s` so its characters appear in the order given by string `order`.	Count chars in `s`, emit them in `order`'s sequence, then append any chars not mentioned in `order`.	O(n)	O(n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
794	Valid Tic-Tac-Toe State	Decide whether the given tic-tac-toe board is reachable from valid play.	Count X and O; X count must equal O or be one more, and if a player has won the move counts must be consistent (both can't win).	O(1)	O(1)	Standard
796	Rotate String	Check if string `t` is a rotation of string `s`.	Every rotation of `s` is a substring of `s+s`, so check containment after a length match.	$O(n^2)$	O(n)	Standard
809	Expressive Words	Determine how many query words match a stretched version of `s` (groups stretched to length ≥ 3).	Compare run lengths group by group: the stretched group must be ≥ 3, or equal to the original run length.	O(n·k)	O(1)	Standard
824	Goat Latin	Convert words to Goat Latin: append "ma" (plus per-word index of 'a's); move leading consonants to the end for non-vowel words.	For each word, decide vowel vs consonant start, transform, then append "ma" and i+1 trailing 'a's.	O(n)	O(n)	Standard
833	Find And Replace in String	Apply non-overlapping indexed replacements: at each index, if `sources[k]` matches, replace it with `targets[k]`.	Map each start index to its replacement; rebuild the string, jumping over matched sources and copying unmatched chars verbatim.	O(n)	O(n)	Standard
859	Buddy Strings	Check if two strings are "buddy strings": swapping exactly one pair of chars makes them equal.	Equal length is required; if the strings are identical, you need a duplicate char to swap; otherwise exactly two positions must differ and be mirror images.	O(n)	O(1)	Standard
953	Verifying an Alien Dictionary	Check if words are sorted in a given alien language's alphabetical order.	Map each letter to its rank, then verify each adjacent pair is in non-decreasing order under that ranking.	O(n·k)	O(1)	Standard
1055	Shortest Way to Form String	Find the minimum number of subsequences of `source` whose concatenation equals `target`. Return -1 if impossible.	Greedily consume as much of `target` as possible from each pass over `source`; if a pass makes no progress, a needed char is missing.	O(n·m)	O(1)	Standard
1108	Defanging an IP Address	Defang an IP address by replacing each '.' with '[.]'.	Append each char, expanding dots into the bracketed form.	O(n)	O(n)	Standard
1221	Split a String in Balanced Strings	Find the maximum number of balanced strings ('R' count == 'L' count) to split into.	Track a running balance; every time it returns to zero a balanced piece has closed.	O(n)	O(1)	Standard
1328	Break a Palindrome	Break a palindrome by changing one character to get the lexicographically smallest non-palindrome.	Change the first non-'a' in the first half to 'a'; if all are 'a', change the last char to 'b'. Single-char strings can't be broken.	O(n)	O(n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
1392	Longest Happy Prefix	Find the longest happy prefix (a non-empty prefix that is also a suffix).	The KMP failure value at the last index is exactly the length of the longest proper prefix that is also a suffix.	O(n)	O(n)	Standard
1446	Consecutive Characters	Find the maximum number of consecutive same characters in a string (the "power" of the string).	Track the current run length, resetting on a change, and keep the maximum seen.	O(n)	O(1)	Standard
1554	Strings Differ by One Character	Check if any two strings in a list differ by exactly one character (same length, same position).	For each position, hash each word with that position wildcarded; a collision among same-length words means they differ in only that spot.	$O(n \cdot m^2)$	$O(n \cdot m)$	Standard
1556	Thousand Separator	Format an integer with a dot as the thousands separator.	Build the result from the right, inserting a separator after every three digits.	O(d)	O(d)	Standard
1736	Latest Time by Replacing Hidden Digits	Find the latest valid time by replacing each '?' with an appropriate digit.	Greedily choose the largest valid digit at each '?' position, respecting hour (≤ 23) and minute (≤ 59) constraints.	O(1)	O(1)	Standard
1816	Truncate Sentence	Truncate a sentence to its first `k` words.	Copy chars until the (k)th space is reached.	O(n)	O(n)	Standard
1844	Replace All Digits with Characters	Replace each digit at an odd index with the letter shifted from the preceding char by that digit.	Even indices are letters; for each odd index, shift the previous letter forward by the digit's value.	O(n)	O(n)	Standard

---

Core Idioms

Variables: `count` = frequency vector (index = char/digit bucket) · `ch - 'a'` = 0–25 bucket index · `ch - '0'` = 0–9 digit value · `num` = number built so far · `builder` = encoded/result string · `buckets` = lists indexed by frequency · `expand(i,j)` = palindrome length from a center **Pseudocode:**

```
idiom 1 — char count for letters:
  make count of size 26
  for each char: count[ch-'a'] += 1
idiom 2 — char count for digits:
  make count of size 10
  for each char: count[ch-'0'] += 1
idiom 3 — char to integer:
  num = 0
  for each char left to right: num = num*10 + (char - '0')
idiom 4 — anagram hash key (frequency-based):
  count the 26 letters of s
  build a string from each letter label followed by its count
  return it (same key for all anagrams)
  alternative: sort the chars and use the sorted string as key
idiom 5 — bucket sort by frequency:
  count all ASCII chars
  make buckets indexed by frequency
  for each char with count>0: add it to buckets[its count]
  walk buckets from highest frequency down, append each char that many times
idiom 6 — expand from center:
  while i in bounds and j in bounds and chars at i and j match: move i left, j right
  return j - i - 1 (length of palindrome found)
```

```java
// a lowercase string is a length-26 frequency vector; most string problems are vector ops on it.  — WH
// 1. CHAR COUNT — lowercase letters
int[] count = new int[26];
// WHY ch - 'a': ASCII 'a'=97, so 'a'→0, 'b'→1, ..., 'z'→25 — a clean 0–25 bucket index into count[].
for (char ch : s.toCharArray()) count[ch - 'a']++;     // ← ch - 'a' maps a→0, b→1, ..., z→25

// 2. CHAR COUNT — digits 0–9
int[] count = new int[10];
for (char ch : s.toCharArray()) count[ch - '0']++;     // ← ch - '0' maps '0'→0, '9'→9

// 3. CHAR TO INTEGER — build number digit by digit
int num = 0;
for (int i = 0; i < s.length(); i++)
    num = num * 10 + (s.charAt(i) - '0');              // ← shift left × 10, add new digit

// 4. ANAGRAM HASH KEY — frequency-based (bucket sort style) O(k) vs sort O(k log k)
// WHY it works: anagrams share identical letter frequencies, so encoding the frequency vector
//   ITSELF gives a unique key for the whole group — O(k) to build vs O(k log k) to sort the chars.
String hashKey(String s) {
    int[] count = new int[26];
    for (char ch : s.toCharArray()) count[ch - 'a']++;
    StringBuilder builder = new StringBuilder();
    for (int i = 0; i < 26; i++) {
        builder.append((char)('a' + i));                    // ← letter as label
        builder.append(count[i]);                           // ← its count
    }
    return builder.toString();                              // e.g., "a2b0c1...z0"
}
// Alternative: sort the characters (simpler, slightly slower)
char[] chars = s.toCharArray();
Arrays.sort(chars);
String key = new String(chars);                        // anagrams → same sorted key

// 5. BUCKET SORT — sort by frequency without comparison sort
int[] count = new int[128];                            // ASCII
for (char ch : s.toCharArray()) count[ch]++;
List<Character>[] buckets = new List[s.length() + 1]; // index = frequency
for (int i = 0; i < 128; i++) {
    if (count[i] > 0) {
        int freq = count[i];
        if (buckets[freq] == null) buckets[freq] = new ArrayList<>();
        buckets[freq].add((char) i);
    }
}
StringBuilder builder = new StringBuilder();
for (int freq = buckets.length - 1; freq > 0; freq--) {    // ← descending frequency
    if (buckets[freq] == null) continue;
    for (char ch : buckets[freq])
        for (int i = 0; i < freq; i++) builder.append(ch);
}

// 6. EXPAND FROM CENTER — palindrome check in O(1) space
int expand(String s, int i, int j) {          // i == j: odd length; i+1 == j: even length
    while (i >= 0 && j < s.length() && s.charAt(i) == s.charAt(j)) { i--; j++; }
    return j - i - 1;                         // ← length of palindrome found
}
```

Part 1 — Char Frequency

#242 Valid Anagram

Description: Return true if `t` is an anagram of `s` (same characters, same counts).
Intuition: Anagrams have identical letter counts, so add up `s` and subtract `t` in one 26-slot array — all zeros means they match.

Variables: `count[ch-'a']` = net frequency of each letter (s adds, t subtracts) · `c` = one slot's net count **Pseudocode:**

```
if lengths differ: return false
make count of size 26
for each char in s: add 1 to count[ch-'a']
for each char in t: subtract 1 from count[ch-'a']
for each slot c: if c is non-zero, return false
return true
```

```
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] count = new int[26];
        for (char ch : s.toCharArray()) count[ch - 'a']++;    // ← fill from s
        for (char ch : t.toCharArray()) count[ch - 'a']--;    // ← drain from t
        for (int c : count) if (c != 0) return false;         // ← any non-zero → not anagram
        return true;
    }
}
```

Time O(n) | **Space** O(1)

#383 Ransom Note

Description: Can `ransomNote` be constructed from the letters in `magazine` (each letter used at most once)?
Intuition: Stock the letter counts from the magazine, then draw down for each note letter — if any count goes negative the magazine is short.

Variables: `count[ch-'a']` = available copies of each letter from the magazine **Pseudocode:**

```
make count of size 26
for each char in magazine: add 1 to count[ch-'a']
for each char in ransomNote: subtract 1 from count[ch-'a'], and if it drops below 0 return false
return true
```

```
class Solution {
    public boolean canConstruct(String ransomNote, String magazine) {
        int[] count = new int[26];
        for (char ch : magazine.toCharArray())   count[ch - 'a']++;     // ← fill from magazine
        for (char ch : ransomNote.toCharArray()) if (--count[ch - 'a'] < 0) return false; // ← drain
        return true;
    }
}
```

Time O(n) | **Space** O(1)

#387 First Unique Character in a String

Description: Return the index of the first non-repeating character. Return -1 if none exists.
Intuition: One pass to count every letter, a second pass to return the first whose count is exactly 1.

Variables: `count[ch-'a']` = frequency of each letter · `i` = scan index **Pseudocode:**

```
make count of size 26
for each char in s: add 1 to count[ch-'a']
for i from 0 to end of s:
  if count of the char at i equals 1: return i
return -1
```

```
class Solution {
    public int firstUniqChar(String s) {
        int[] count = new int[26];
        for (char ch : s.toCharArray()) count[ch - 'a']++;
        for (int i = 0; i < s.length(); i++)
            if (count[s.charAt(i) - 'a'] == 1) return i;    // ← first with count exactly 1
        return -1;
    }
}
```

Time O(n) | **Space** O(1)

Part 2 — Anagram Hashing & Bucket Sort

#49 Group Anagrams

Description: Group strings that are anagrams of each other.
Intuition: Anagrams share identical letter frequencies, so the frequency vector encoded as a string is a unique group key — O(k) to build vs O(k log k) to sort the characters.
Key: each anagram group shares the same frequency hash key. `computeIfAbsent` cleanly handles first-time group creation.

Variables: `map` = key → list of strings sharing that key · `key` = frequency-vector string (same for all anagrams) · `count[ch-'a']` = letter frequency · `builder` = the key being built **Pseudocode:**

```
make empty map from key to list
for each string s:
  key = hashKey(s)
  add s to map[key] (creating the list if absent)
return all the lists in the map

hashKey(s):
  count the 26 letters of s
  build a string from each letter label followed by its count
  return that string
```

```java
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            String key = hashKey(s);                                    // ← frequency-based key
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);   // ← group by key
        }
        return new ArrayList<>(map.values());
    }

    private String hashKey(String s) {                                  // O(k) — no sort needed
        int[] count = new int[26];
        for (char ch : s.toCharArray()) count[ch - 'a']++;             // ← count each char
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < 26; i++) {
            builder.append((char)('a' + i));                                  // ← letter label
            builder.append(count[i]);                                         // ← its count
        }
        return builder.toString();                                            // e.g., "a1e1t1"
    }
}
// Time O(n·k) | Space O(n)   — k = avg string length
```

Alternative key (sort-based, simpler but O(k log k)):

Variables: `chars` = the string's characters · `key` = the sorted characters as a string (same for all anagrams)
Pseudocode:

```
turn s into a char array
sort the chars
the sorted string is the key
```

```java
char[] chars = s.toCharArray();
Arrays.sort(chars);
String key = new String(chars);   // "eat" → "aet", "tea" → "aet"
```

#451 Sort Characters By Frequency

Description: Sort characters in descending order of frequency. Ties can go in any order.
Intuition: Frequencies are bounded by string length, so bucket chars by their count and read buckets high-to-low — no O(n log n) comparison sort needed.
Key: bucket sort by frequency — avoids O(n log n) comparison sort. Each bucket holds all chars with that frequency.

Variables: `count[ch]` = frequency of each ASCII char · `buckets[freq]` = list of chars that occur exactly `freq` times · `builder` = result string **Pseudocode:**

```
count the frequency of every ASCII char
make buckets indexed by frequency (0..length)
for each char with count>0: add it to buckets[its count]
for freq from length down to 1:
  if that bucket is empty, skip
  for each char in the bucket: append it freq times
return the result
```

```java
class Solution {
    public String frequencySort(String s) {
        int[] count = new int[128];                                 // ASCII range
        for (char ch : s.toCharArray()) count[ch]++;

        List<Character>[] buckets = new List[s.length() + 1];          // ← bucket index = frequency
        for (int i = 0; i < 128; i++) {
            if (count[i] > 0) {
                int freq = count[i];
                if (buckets[freq] == null) buckets[freq] = new ArrayList<>();
                buckets[freq].add((char) i);
            }
        }

        StringBuilder builder = new StringBuilder();
        for (int freq = s.length(); freq > 0; freq--) {                // ← descending frequency
            if (buckets[freq] == null) continue;
            for (char ch : buckets[freq])
                for (int i = 0; i < freq; i++) builder.append(ch);          // ← repeat char `freq` tim
        }
        return builder.toString();
    }
}
// Time O(n) | Space O(n)
```

Part 3 — Palindrome (Expand from Center)

Template

Variables: `i` = center index · `(i,i)` = odd-length center · `(i,i+1)` = even-length center · `expand(i,j)` = length of palindrome grown from that center **Pseudocode:**

```
for each index i in s:
  process the odd center (i, i)
  process the even center (i, i+1)

expand(i, j):
  while i and j are in bounds and the chars match: move i left, j right
  return j − i − 1 (the palindrome length)
```

```java
// Call for each center: odd-length palindromes (i == i) and even-length (i, i+1)
for (int i = 0; i < s.length(); i++) {
    // odd:  single center at i
    // even: center between i and i+1
    processExpansion(s, i, i);     // odd
    processExpansion(s, i, i + 1); // even
}

// Shared expand helper — returns length of palindrome
int expand(String s, int i, int j) {
    while (i >= 0 && j < s.length() && s.charAt(i) == s.charAt(j)) { i--; j++; }
    return j - i - 1;
}
```

#5 Longest Palindromic Substring

Description: Return the longest palindromic substring.
Intuition: Every palindrome has a center; try all 2n-1 centers (each char and each gap) and grow outward while characters mirror.
Key: expand from every center (odd and even), track the longest found.

Variables: `start` = start index of best palindrome · `maxLen` = best length so far · `odd` / `even` = palindrome lengths from the two center types · `len` = best of the two · `expand(i,j)` = palindrome length from a center **Pseudocode:**

```
start = 0, maxLen = 1
for each index i:
  odd = expand from center (i, i)
  even = expand from center (i, i+1)
  len = the larger of odd and even
  if len beats maxLen: update maxLen and back-compute start = i − (len−1)/2
return the substring from start of length maxLen

expand(i, j):
  while in bounds and chars match: move i left, j right
  return j − i − 1
```

```
class Solution {
    public String longestPalindrome(String s) {
        int n = s.length(), start = 0, maxLen = 1;
        for (int i = 0; i < n; i++) {
            int odd  = expand(s, i, i);                        // odd-length palindrome
            int even = expand(s, i, i + 1);                    // even-length palindrome
            int len  = Math.max(odd, even);
            if (len > maxLen) {
                maxLen = len;
                start  = i - (len - 1) / 2;                   // ← back-compute start from center
            }
        }
        return s.substring(start, start + maxLen);
    }
    private int expand(String s, int i, int j) {
        while (i >= 0 && j < s.length() && s.charAt(i) == s.charAt(j)) { i--; j++; }
        return j - i - 1;
    }
}
```

Time O(n^2) | **Space** O(1)

#647 Palindromic Substrings

Description: Count all palindromic substrings.
Intuition: Same center-expansion, but each successful step outward is itself one more palindrome, so accumulate a count rather than a max.
Key: same expand helper — count expansions instead of tracking max length.

Variables: `count` = total palindromes found · `expandCount(i,j)` = number of palindromes centered there (one per successful expansion step) **Pseudocode:**

```
count = 0
for each index i:
  count += palindromes from odd center (i, i)
  count += palindromes from even center (i, i+1)
return count

expandCount(i, j):
  count = 0
  while in bounds and chars match: count += 1, move i left, j right
  return count
```

```
class Solution {
    public int countSubstrings(String s) {
        int count = 0;
        for (int i = 0; i < s.length(); i++) {
            count += expandCount(s, i, i);       // odd: each expansion = 1 new palindrome
            count += expandCount(s, i, i + 1);   // even
        }
        return count;
    }
    private int expandCount(String s, int i, int j) {
        int count = 0;
        while (i >= 0 && j < s.length() && s.charAt(i) == s.charAt(j)) {
            count++;                             // ← each valid expansion = 1 palindrome
            i--; j++;
        }
        return count;
    }
}
```

Time O(n^2) | **Space** O(1)

#5 vs #647 — Same expand, different accumulation

```
                    #5 Longest Palindromic     #647 Palindromic Substrings
Per expansion:      track max length           count++
Outer loop:         track best start+len       accumulate count
Return:             s.substring(start, start+len)    count
Expand returns:     palindrome length          palindrome count
```

Part 4 — String Parsing & Building

#8 String to Integer (atoi)

Description: Parse a string to an integer, handling leading spaces, optional sign, overflow, and non-digit stop.
Intuition: Process the string in strict phases — spaces, then sign, then digits accumulated as `num*10 + digit` — clamping to int bounds the moment overflow would occur.
Key steps in order: skip spaces → read sign → read digits (`num = num * 10 + (ch - '0')`) → clamp on overflow.

Variables: `i` = scan index · `sign` = +1 or -1 · `result` = number built so far (long, to detect overflow) **Pseudocode:**

```
i = 0
skip leading spaces
if reached end: return 0
sign = +1; if next is '-' set -1 and advance, if '+' just advance
result = 0
while next is a digit:
  result = result*10 + digit
  if result*sign exceeds int max: return int max
  if result*sign below int min: return int min
return result*sign as int
```

```
class Solution {
    public int myAtoi(String s) {
        int i = 0, n = s.length();
        while (i < n && s.charAt(i) == ' ') i++;                  // ← 1. skip leading spaces
        if (i == n) return 0;
        int sign = 1;
        if      (s.charAt(i) == '-') { sign = -1; i++; }          // ← 2. read sign
        else if (s.charAt(i) == '+') { i++; }
        long result = 0;
        while (i < n && Character.isDigit(s.charAt(i))) {
            result = result * 10 + (s.charAt(i++) - '0');          // ← 3. digit by digit
            if (result * sign >  Integer.MAX_VALUE) return Integer.MAX_VALUE; // ← 4. clamp
            if (result * sign <  Integer.MIN_VALUE) return Integer.MIN_VALUE;
        }
        return (int)(result * sign);
    }
}
```

Time O(n) | **Space** O(1)

#14 Longest Common Prefix

Description: Find the longest common prefix string among an array of strings.
Intuition: Start with the first string as the candidate prefix and trim its tail until every other string starts with it.
Key: use the first string as the candidate prefix. Shrink it until every string starts with it.

Variables: `prefix` = current candidate common prefix **Pseudocode:**

```
prefix = the first string
for each other string:
  while that string does not start with prefix: drop prefix's last char
  if prefix became empty: return ""
return prefix
```

```
class Solution {
    public String longestCommonPrefix(String[] strs) {
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (!strs[i].startsWith(prefix))                    // ← shrink until prefix matches
                prefix = prefix.substring(0, prefix.length() - 1);
            if (prefix.isEmpty()) return "";
        }
        return prefix;
    }
}
```

Time O(n·k) | **Space** O(1)

#443 String Compression

Description: Compress the char array in-place. Each group `aaa` becomes `a3`. Single chars stay as-is. Return new length.
Intuition: Two pointers — read scans each run while write lays down the compressed `char + count` behind it; write never overtakes read, so it's safe in place.
Key: write pointer `write` advances independently of read pointer `i`. Count run length, then write char + (count if > 1).

Variables: `write` = next position to write the compressed output · `i` = read scan index · `ch` = current run's char · `count` = run length **Pseudocode:**

```
write = 0, i = 0
while i < length:
  ch = chars[i]
  count consecutive copies of ch, advancing i
  write ch at the write position and advance write
  if count > 1: write each digit of count, advancing write
return write (the new length)
```

```
class Solution {
    public int compress(char[] chars) {
        int write = 0, i = 0;
        while (i < chars.length) {
            char ch = chars[i];
            int count = 0;
            while (i < chars.length && chars[i] == ch) { i++; count++; }  // ← count run
            chars[write++] = ch;                                            // ← write char
            if (count > 1)
                for (char c : Integer.toString(count).toCharArray())        // ← write digits
                    chars[write++] = c;
        }
        return write;
    }
}
```

Time O(n) | **Space** O(1)

#1047 Remove All Adjacent Duplicates in String

Description: Repeatedly remove adjacent equal characters until no more exist.
Intuition: A stack mirrors the partially-cleaned string — if the next char equals the top it's an adjacent pair, so pop instead of pushing.
Key: stack — push each char; if it equals the top, they form a pair → pop instead. Stack holds the result after all removals.

Variables: `stack` = chars of the partially-cleaned string · `ch` = current char · `builder` = final result **Pseudocode:**

```
make empty stack
for each char ch:
  if stack not empty and top equals ch: pop (the pair cancels)
  else: push ch
pop everything into a builder, then reverse it
return the result
```

```java
class Solution {
    public String removeDuplicates(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char ch : s.toCharArray()) {
            if (!stack.isEmpty() && stack.peek() == ch) {
                stack.pop();  // ← pair → cancel
            } else {
                stack.push(ch);
            }
        }
        StringBuilder builder = new StringBuilder();
        while (!stack.isEmpty()) builder.append(stack.pop());
        return builder.reverse().toString();
    }
}
```

Time O(n) | **Space** O(n)

String Idioms Cheat Sheet

Variables: `ch` = a single char · `i` = an index/offset · `s` = a string · `builder` = a StringBuilder accumulating output · `map` = a grouping map **Pseudocode:**

```
char/index conversions: 'a'+i makes a letter, ch-'a' makes a letter index, ch-'0' makes a digit; test/cor
string operations: read a char, convert to/from char array, take a substring, test contains/startsWith, s
building strings: append chars or strings to a builder, delete or reverse, then convert to string, or joi
grouping: use computeIfAbsent to fetch-or-create a list for a key, then add the value
```

```
// Char ↔ index
'a' + i             // int → char (i=0→'a', i=25→'z')
ch - 'a'            // char → index (a→0, z→25)
ch - '0'            // char → digit (0→0, 9→9)
Character.isDigit(ch)
Character.isLetter(ch)
Character.toLowerCase(ch)

// String operations
s.charAt(i)                      // char at index
s.toCharArray()                  // → char[]
new String(charArray)            // char[] → String
String.valueOf(charArray)        // char[] → String
s.substring(i, j)                // [i, j)
s.contains(sub)                  // boolean
s.startsWith(prefix)             // boolean
s.split("\\s+")                  // split on whitespace

// Building strings
StringBuilder builder = new StringBuilder();
builder.append(ch);         builder.append(str);
builder.deleteCharAt(i);
builder.reverse();
builder.toString();
String.join(", ", list)          // join with delimiter

// computeIfAbsent — key pattern for grouping
map.computeIfAbsent(key, k -> new ArrayList<>()).add(value);
```

Pattern Chooser

Signal	Pattern
Are two strings anagrams?	Count array: fill from s, drain from t
Group strings by anagram	Frequency hash key (`hashKey`) or `Arrays.sort` key
Sort/rank by character frequency	Bucket sort: count[], then buckets[freq]
Longest/count palindromic substrings	Expand from center (i,i) odd + (i,i+1) even
Parse a number from string	`num = num * 10 + (ch - '0')`
Remove adjacent duplicate pairs	Stack: push, pop on match
Sliding window over characters	See `sliding_window.md`

Additional Reference Problems

#6 ZigZag Conversion

Description: Convert a string into a zigzag pattern across `numRows` rows and read it row by row.
Intuition: Walk the string once, appending each char to its current row's builder while bouncing the row index down then up.

Variables: `rows[i]` = builder for the i-th zigzag row · `row` = current row index · `step` = +1 going down, -1 going up · `result` = rows concatenated **Pseudocode:**

```
if there is only one row: return s unchanged
make one builder per row
row = 0, step = 1 (downward)
for each char:
  append it to the current row's builder
  if at the top row: step = down; if at the bottom row: step = up
  move row by step
concatenate all row builders and return
```

```java
class Solution {
    public String convert(String s, int numRows) {
        if (numRows == 1) {
            return s;
        }
        StringBuilder[] rows = new StringBuilder[numRows];
        for (int i = 0; i < numRows; i++) {
            rows[i] = new StringBuilder();
        }
        int row = 0, step = 1;
        for (char ch : s.toCharArray()) {
            rows[row].append(ch);
            if (row == 0) {
                step = 1;                         // ← bounce down
            } else if (row == numRows - 1) {
                step = -1;                        // ← bounce up
            }
            row += step;
        }
        StringBuilder result = new StringBuilder();
        for (StringBuilder builder : rows) {
            result.append(builder);
        }
        return result.toString();
    }
}
```

Time O(n) | **Space** O(n)

#12 Integer to Roman

Description: Convert an integer to its Roman numeral representation (values 1–3999).
Intuition: Greedily subtract the largest possible value (including the 4/9 subtractive pairs) and append its symbol.

Variables: `values` = Roman values high to low (incl. 900/400/90/...) · `symbols` = matching symbols · `builder` = result · `num` = remaining amount **Pseudocode:**

```
list the values high to low and their symbols (including the subtractive pairs)
for each value/symbol pair:
  while num is at least this value: append the symbol and subtract the value
return the result
```

```java
class Solution {
    public String intToRoman(int num) {
        int[] values =    {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
        String[] symbols = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"};
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < values.length; i++) {
            while (num >= values[i]) {
                builder.append(symbols[i]);        // ← append largest fitting symbol
                num -= values[i];
            }
        }
        return builder.toString();
    }
}
```

Time O(1) | **Space** O(1)

#13 Roman to Integer

Description: Convert a Roman numeral string to an integer.
Intuition: Add each symbol's value, but subtract instead when a smaller value precedes a larger one (subtractive notation).

Variables: `map` = symbol → value · `result` = running total · `value` = current symbol's value **Pseudocode:**

```
map each Roman symbol to its value
result = 0
for each index i:
  value = the value of symbol at i
  if a larger value follows: subtract value (subtractive pair)
  else: add value
return result
```

```java
class Solution {
    public int romanToInt(String s) {
        Map<Character, Integer> map = new HashMap<>();
        map.put('I', 1); map.put('V', 5); map.put('X', 10); map.put('L', 50);
        map.put('C', 100); map.put('D', 500); map.put('M', 1000);
        int result = 0;
        for (int i = 0; i < s.length(); i++) {
            int value = map.get(s.charAt(i));
            if (i + 1 < s.length() && value < map.get(s.charAt(i + 1))) {
                result -= value;                  // ← subtractive pair (IV, IX, ...)
            } else {
                result += value;
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#28 Implement strStr()

Description: Find the first occurrence of `needle` in `haystack`. Return -1 if not found.
Intuition: Slide a window of needle's length across haystack and compare; the first full match wins.

Variables: `n` / `m` = lengths of haystack/needle · `i` = window start in haystack · `j` = match offset within needle
Pseudocode:

```
for each start i where the needle still fits:
  j = 0
  while j < m and haystack[i+j] equals needle[j]: advance j
  if j reached m: full match, return i
return -1
```

```
class Solution {
    public int strStr(String haystack, String needle) {
        int n = haystack.length(), m = needle.length();
        for (int i = 0; i + m <= n; i++) {
            int j = 0;
            while (j < m && haystack.charAt(i + j) == needle.charAt(j)) {
                j++;
            }
            if (j == m) {
                return i;                          // ← full needle matched
            }
        }
        return -1;
    }
}
```

Time O(n·m) | **Space** O(1)

#38 Count and Say

Description: Generate the nth term of the "count and say" sequence: read digits of the previous term aloud.
Intuition: Starting from "1", repeatedly scan runs of equal digits and emit count followed by the digit.

Variables: `result` = current term · `builder` = next term · `j` = scan index · `ch` = current run's digit · `count` = run length **Pseudocode:**

```
result = "1"
repeat n-1 times:
  j = 0, start a fresh builder
  while j < length of result:
    ch = digit at j
    count consecutive copies of ch, advancing j
    append count then ch
  result = builder
return result
```

```java
class Solution {
    public String countAndSay(int n) {
        String result = "1";
        for (int i = 1; i < n; i++) {
            StringBuilder builder = new StringBuilder();
            int j = 0;
            while (j < result.length()) {
                char ch = result.charAt(j);
                int count = 0;
                while (j < result.length() && result.charAt(j) == ch) {  // ← run length
                    count++;
                    j++;
                }
                builder.append(count).append(ch);
            }
            result = builder.toString();
        }
        return result;
    }
}
```

Time O(n·m) | **Space** O(m)

#43 Multiply Strings

Description: Multiply two non-negative integers given as strings. Return the product as a string.
Intuition: Schoolbook multiplication: digit i of num1 times digit j of num2 lands in result positions i+j and i+j+1.

Variables: `product[k]` = digit accumulator for result position k · `mul` = single-digit product · `sum` = mul plus what's already at i+j+1 · `builder` = trimmed result **Pseudocode:**

```
if either number is "0": return "0"
make product array of size n+m
for i from last digit of num1 down:
  for j from last digit of num2 down:
    mul = digit i times digit j
    sum = mul + product[i+j+1]
    product[i+j+1] = sum mod 10 (low digit)
    product[i+j] += sum / 10 (carry up)
build a string from product, skipping leading zeros
return it
```

```java
class Solution {
    public String multiply(String num1, String num2) {
        if (num1.equals("0") || num2.equals("0")) {
            return "0";
        }
        int n = num1.length(), m = num2.length();
        int[] product = new int[n + m];
        for (int i = n - 1; i >= 0; i--) {
            for (int j = m - 1; j >= 0; j--) {
                int mul = (num1.charAt(i) - '0') * (num2.charAt(j) - '0');
                int sum = mul + product[i + j + 1];
                product[i + j + 1] = sum % 10;     // ← low digit
                product[i + j] += sum / 10;        // ← carry to higher position
            }
        }
        StringBuilder builder = new StringBuilder();
        for (int digit : product) {
            if (!(builder.length() == 0 && digit == 0)) {  // ← skip leading zeros
                builder.append(digit);
            }
        }
        return builder.toString();
    }
}
```

Time O(n·m) | **Space** O(n+m)

#58 Length of Last Word

Description: Return the length of the last word in a string (word = max sequence of non-space chars).

Intuition: Scan from the right: skip trailing spaces, then count letters until the next space.

Variables: `i` = scan index from the right · `length` = length of last word **Pseudocode:**

```
i = last index
skip trailing spaces (decrement i while space)
length = 0
while i >= 0 and char is not space: length += 1, decrement i
return length
```

```java
class Solution {
    public int lengthOfLastWord(String s) {
        int i = s.length() - 1;
        while (i >= 0 && s.charAt(i) == ' ') {     // ← skip trailing spaces
            i--;
        }
        int length = 0;
        while (i >= 0 && s.charAt(i) != ' ') {     // ← count last word
            length++;
            i--;
        }
        return length;
    }
}
```

Time O(n) | **Space** O(1)

#65 Valid Number

Description: Validate whether a string is a valid decimal number (integers, fractions, exponents).
Intuition: A single left-to-right scan tracking whether we have seen a digit, a dot, and an exponent enforces all the ordering rules.

Variables: `seenDigit` = a digit appeared (resets after `e`) · `seenDot` = a `.` appeared · `seenExp` = an `e` / `E` appeared · `ch` = current char **Pseudocode:**

```
seenDigit = seenDot = seenExp = false
for each char ch:
  if digit: seenDigit = true
  else if sign: valid only at start or right after e, else fail
  else if dot: fail if a dot or exponent already seen, else seenDot = true
  else if e/E: fail if exponent already seen or no digit yet; seenExp = true; seenDigit = false (need dig
  else: fail
return seenDigit
```

```
class Solution {
    public boolean isNumber(String s) {
        boolean seenDigit = false, seenDot = false, seenExp = false;
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (Character.isDigit(ch)) {
                seenDigit = true;
            } else if (ch == '+' || ch == '-') {
                if (i > 0 && s.charAt(i - 1) != 'e' && s.charAt(i - 1) != 'E') {
                    return false;                // ← sign only at start or after e
                }
            } else if (ch == '.') {
                if (seenDot || seenExp) {
                    return false;
                }
                seenDot = true;
            } else if (ch == 'e' || ch == 'E') {
                if (seenExp || !seenDigit) {
                    return false;
                }
                seenExp = true;
                seenDigit = false;               // ← need digits after e
            } else {
                return false;
            }
        }
        return seenDigit;
    }
}
```

Time O(n) | **Space** O(1)

#68 Text Justification

Description: Format words into lines of `maxWidth`, fully justified (equal spaces, last line left-aligned).
Intuition: Greedily pack as many words as fit per line, then distribute leftover spaces evenly between gaps (extras to the left); the last line is left-justified.

Variables: `i` / `j` = first/after-last word index of the current line · `lineLen` = total word chars on the line · `wordCount` = words on the line · `spaces` = leftover spaces · `gaps` = word gaps · `base` / `extra` = even spaces per gap and remainder · `builder` = the line **Pseudocode:**

```
i = 0
while i < number of words:
  j = i, lineLen = 0
  greedily add words while they plus the minimum single spaces still fit; track lineLen and advance j
  wordCount = j − i, spaces = maxWidth − lineLen
  if this is the last line or only one word: left-justify (single spaces between, pad right to maxWidth)
  else:
    gaps = wordCount − 1, base = spaces/gaps, extra = spaces mod gaps
    for each word but the last: append word, then base spaces plus one extra for the leftmost `extra` gap
  add the line; set i = j
return all lines
```

```java
class Solution {
    public List<String> fullJustify(String[] words, int maxWidth) {
        List<String> result = new ArrayList<>();
        int i = 0, n = words.length;
        while (i < n) {
            int j = i, lineLen = 0;
            while (j < n && lineLen + words[j].length() + (j - i) <= maxWidth) {
                lineLen += words[j].length();      // ← greedily fit words
                j++;
            }
            StringBuilder builder = new StringBuilder();
            int wordCount = j - i;
            int spaces = maxWidth - lineLen;
            if (j == n || wordCount == 1) {        // ← last line or single word: left-justify
                for (int k = i; k < j; k++) {
                    builder.append(words[k]);
                    if (k < j - 1) {
                        builder.append(' ');
                    }
                }
                while (builder.length() < maxWidth) {
                    builder.append(' ');
                }
            } else {
                int gaps = wordCount - 1;
                int base = spaces / gaps, extra = spaces % gaps;
                for (int k = i; k < j; k++) {
                    builder.append(words[k]);
                    if (k < j - 1) {
                        int pad = base + (k - i < extra ? 1 : 0);  // ← extras to left gaps
                        for (int p = 0; p < pad; p++) {
                            builder.append(' ');
                        }
                    }
                }
            }
            result.add(builder.toString());
            i = j;
        }
        return result;
    }
}
```

Time O(n·maxWidth) | **Space** O(n)

#125 Valid Palindrome

Description: Check if a string is a palindrome, ignoring non-alphanumeric characters and case.
Intuition: Two pointers move inward, skipping non-alphanumeric chars and comparing lowercased letters.

Variables: `i / j` = left/right pointers moving inward **Pseudocode:**

```
i = 0, j = last index
while i < j:
  advance i past non-alphanumeric chars
  retreat j past non-alphanumeric chars
  if lowercased chars at i and j differ: return false
  move i right, j left
return true
```

```
class Solution {
    public boolean isPalindrome(String s) {
        int i = 0, j = s.length() - 1;
        while (i < j) {
            while (i < j && !Character.isLetterOrDigit(s.charAt(i))) {
                i++;                               // ← skip non-alphanumeric
            }
            while (i < j && !Character.isLetterOrDigit(s.charAt(j))) {
                j--;
            }
            if (Character.toLowerCase(s.charAt(i)) != Character.toLowerCase(s.charAt(j))) {
                return false;
            }
            i++;
            j--;
        }
        return true;
    }
}
```

Time O(n) | **Space** O(1)

#151 Reverse Words in a String

Description: Reverse the order of words in a string (split on spaces, handle multiple spaces).
Intuition: Scan from the right, extract each word, and append them in reverse order with single spaces.

Variables: `result` = reversed sentence being built · `i` = scan index from the right · `j` = end index of the current word
Pseudocode:

```
i = last index
while i >= 0:
  skip spaces (decrement i)
  if i < 0: stop
  j = i; scan left over the word (decrement i) until a space or start
  if result already has words: append a space
  append the word spanning i+1..j
return result
```

```java
class Solution {
    public String reverseWords(String s) {
        StringBuilder result = new StringBuilder();
        int i = s.length() - 1;
        while (i >= 0) {
            while (i >= 0 && s.charAt(i) == ' ') {  // ← skip spaces
                i--;
            }
            if (i < 0) {
                break;
            }
            int j = i;
            while (i >= 0 && s.charAt(i) != ' ') {  // ← span a word
                i--;
            }
            if (result.length() > 0) {
                result.append(' ');
            }
            result.append(s, i + 1, j + 1);
        }
        return result.toString();
    }
}
```

Time O(n) | **Space** O(n)

#161 One Edit Distance

Description: Determine if two strings are one edit distance apart (insert, delete, or replace one char).

Intuition: Scan to the first mismatch; equal lengths force a replace, otherwise skip one char in the longer string and require the rest to match.

Variables: `n` / `m` = lengths of s/t · `i` = scan index of the first mismatch **Pseudocode:**

```
if lengths differ by more than 1: return false
for i over the common prefix:
  if chars at i differ:
    if lengths equal: rest after i must match (replace)
    if s longer: s after i+1 must equal t after i (delete from s)
    else: s after i must equal t after i+1 (insert into s)
no mismatch found: true only if lengths differ by exactly 1
```

```java
class Solution {
    public boolean isOneEditDistance(String s, String t) {
        int n = s.length(), m = t.length();
        if (Math.abs(n - m) > 1) {
            return false;
        }
        for (int i = 0; i < Math.min(n, m); i++) {
            if (s.charAt(i) != t.charAt(i)) {
                if (n == m) {
                    return s.substring(i + 1).equals(t.substring(i + 1));  // ← replace
                } else if (n > m) {
                    return s.substring(i + 1).equals(t.substring(i));      // ← delete from s
                } else {
                    return s.substring(i).equals(t.substring(i + 1));      // ← insert into s
                }
            }
        }
        return Math.abs(n - m) == 1;               // ← differ only by trailing char
    }
}
```

Time O(n) | **Space** O(n)

#163 Missing Ranges

Description: Given a sorted array, return missing ranges between `lower` and `upper` bounds.
Intuition: Walk a running "next expected" value; whenever a number exceeds it, the gap before it is a missing range.

Variables: `next` = smallest value not yet covered · `num` = current array element · `format(lo,hi)` = single value or "lo->hi" **Pseudocode:**

```
next = lower
for each num:
  if num > next: record the gap from next to num−1
  next = num + 1
if next <= upper: record the final gap from next to upper
return the ranges

format(lo, hi): if lo equals hi return lo, else return "lo->hi"
```

```
class Solution {
    public List<String> findMissingRanges(int[] nums, int lower, int upper) {
        List<String> result = new ArrayList<>();
        long next = lower;
        for (int num : nums) {
            if (num > next) {
                result.add(format(next, (long) num - 1));   // ← gap before num
            }
            next = (long) num + 1;
        }
        if (next <= upper) {
            result.add(format(next, (long) upper));
        }
        return result;
    }
    private String format(long lo, long hi) {
        return lo == hi ? String.valueOf(lo) : lo + "->" + hi;
    }
}
```

Time O(n) | **Space** O(1)

#165 Compare Version Numbers

Description: Compare two version number strings (dot-separated integers). Return -1, 0, or 1.

Intuition: Parse both revisions position by position, treating missing components as 0, and compare numerically.

Variables: `a` / `b` = dot-split components of each version · `x` / `y` = numeric value of each component (0 if missing)

Pseudocode:

```
split both versions on '.'
for i up to the longer length:
  x = component i of a, or 0 if missing
  y = component i of b, or 0 if missing
  if x != y: return -1 or 1
return 0
```

```
class Solution {
    public int compareVersion(String version1, String version2) {
        String[] a = version1.split("\\.");
        String[] b = version2.split("\\.");
        int n = Math.max(a.length, b.length);
        for (int i = 0; i < n; i++) {
            int x = i < a.length ? Integer.parseInt(a[i]) : 0;   // ← missing → 0
            int y = i < b.length ? Integer.parseInt(b[i]) : 0;
            if (x != y) {
                return x < y ? -1 : 1;
            }
        }
        return 0;
    }
}
```

Time O(n) | **Space** O(n)

#166 Fraction to Recurring Decimal

Description: Convert a fraction numerator/denominator to a decimal string, noting repeating parts in parentheses.
Intuition: Do long division; remember the position of each remainder so a repeat reveals the cycle to wrap in parentheses.

Variables: `builder` = result string · `num` / `den` = absolute numerator/denominator · `remainder` = current long-division remainder · `seen` = remainder → position in builder where its digit started **Pseudocode:**

```
if numerator is 0: return "0"
append '-' if exactly one of numerator/denominator is negative
take absolute values; append integer part num/den; remainder = num mod den
if remainder is 0: return result
append '.'; make empty seen map
while remainder != 0:
  if remainder already in seen: insert '(' at its recorded position, append ')', stop
  record remainder -> current builder length
  remainder *= 10; append remainder/den; remainder = remainder mod den
return result
```

```java
class Solution {
    public String fractionToDecimal(int numerator, int denominator) {
        if (numerator == 0) {
            return "0";
        }
        StringBuilder builder = new StringBuilder();
        if ((numerator < 0) ^ (denominator < 0)) {
            builder.append('-');                    // ← sign
        }
        long num = Math.abs((long) numerator);
        long den = Math.abs((long) denominator);
        builder.append(num / den);                  // ← integer part
        long remainder = num % den;
        if (remainder == 0) {
            return builder.toString();
        }
        builder.append('.');
        Map<Long, Integer> seen = new HashMap<>();
        while (remainder != 0) {
            if (seen.containsKey(remainder)) {      // ← cycle detected
                builder.insert(seen.get(remainder), "(");
                builder.append(")");
                break;
            }
            seen.put(remainder, builder.length());
            remainder *= 10;
            builder.append(remainder / den);
            remainder %= den;
        }
        return builder.toString();
    }
}
```

Time O(den) | **Space** O(den)

#179 Largest Number

Description: Arrange numbers so their concatenation forms the largest possible number.
Intuition: Sort by a custom comparator that prefers the order producing a larger concatenation (a+b vs b+a).

Variables: `strs` = numbers as strings · comparator orders by which of `a+b` vs `b+a` is larger · `builder` = concatenated result **Pseudocode:**

```
convert each number to a string
sort so that for any pair a,b, whichever of (b+a) vs (a+b) is larger comes first
if the largest string is "0": all zeros, return "0"
concatenate all strings and return
```

```
class Solution {
    public String largestNumber(int[] nums) {
        String[] strs = new String[nums.length];
        for (int i = 0; i < nums.length; i++) {
            strs[i] = String.valueOf(nums[i]);
        }
        Arrays.sort(strs, (a, b) -> (b + a).compareTo(a + b));   // ← whichever concat is larger first
        if (strs[0].equals("0")) {
            return "0";                            // ← all zeros
        }
        StringBuilder builder = new StringBuilder();
        for (String s : strs) {
            builder.append(s);
        }
        return builder.toString();
    }
}
```

Time O(n log n · k) | **Space** O(n)

#186 Reverse Words in a String II

Description: Reverse words in a character array in-place (words separated by spaces).
Intuition: Reverse the whole array, then reverse each word back so the order flips but each word reads forward.

Variables: `start` = start index of the current word · `i` = scan index marking word boundaries · `reverse(i,j)` = in-place reverse of a range **Pseudocode:**

```
reverse the entire array
start = 0
for i from 0 to length (inclusive):
  if at end or a space: reverse the word from start to i−1, then set start = i+1

reverse(i, j): swap ends moving inward until i meets j
```

```java
class Solution {
    public void reverseWords(char[] s) {
        reverse(s, 0, s.length - 1);               // ← reverse everything
        int start = 0;
        for (int i = 0; i <= s.length; i++) {
            if (i == s.length || s[i] == ' ') {
                reverse(s, start, i - 1);          // ← fix each word back
                start = i + 1;
            }
        }
    }
    private void reverse(char[] s, int i, int j) {
        while (i < j) {
            char tmp = s[i];
            s[i] = s[j];
            s[j] = tmp;
            i++;
            j--;
        }
    }
}
```

Time O(n) | **Space** O(1)

#205 Isomorphic Strings

Description: Check if two strings follow the same character-substitution pattern (isomorphic).
Intuition: Maintain a bijective mapping both ways; any conflicting mapping breaks isomorphism.

Variables: `mapS[a]` = char that s-char `a` maps to (-1 if unset) · `mapT[b]` = char that t-char `b` maps back to · `a` / `b` = current chars of s/t **Pseudocode:**

```
make mapS and mapT, all set to -1 (unset)
for each index i with a = s[i], b = t[i]:
  if both unset: link a->b and b->a
  else if existing links disagree (mapS[a] != b or mapT[b] != a): return false
return true
```

```java
class Solution {
    public boolean isIsomorphic(String s, String t) {
        int[] mapS = new int[256], mapT = new int[256];
        Arrays.fill(mapS, -1);
        Arrays.fill(mapT, -1);
        for (int i = 0; i < s.length(); i++) {
            char a = s.charAt(i), b = t.charAt(i);
            if (mapS[a] == -1 && mapT[b] == -1) {
                mapS[a] = b;                        // ← establish bijection
                mapT[b] = a;
            } else if (mapS[a] != b || mapT[b] != a) {
                return false;                       // ← conflict
            }
        }
        return true;
    }
}
```

Time O(n) | **Space** O(1)

#214 Shortest Palindrome

Description: Find the shortest palindrome by adding characters in front of a string.
Intuition: Find the longest palindromic prefix via KMP failure on `s + '#' + reverse(s)`; reverse the leftover suffix and prepend it.

Variables: `reversed` = s reversed · `combined` = s + '#' + reversed · `lps[i]` = longest prefix-also-suffix length ending at i · `j` = KMP fallback pointer · `palinLen` = longest palindromic prefix length · `suffix` = leftover to prepend
Pseudocode:

```
reversed = reverse of s
combined = s + "#" + reversed
build KMP lps array over combined:
  for i from 1: j = lps[i−1]; fall back while mismatch; if match j++; lps[i] = j
palinLen = lps[last] (length of longest palindromic prefix of s)
suffix = the first (len(s) − palinLen) chars of reversed
return suffix + s
```

```java
class Solution {
    public String shortestPalindrome(String s) {
        String reversed = new StringBuilder(s).reverse().toString();
        String combined = s + "#" + reversed;
        int[] lps = new int[combined.length()];
        for (int i = 1; i < combined.length(); i++) {
            int j = lps[i - 1];
            while (j > 0 && combined.charAt(i) != combined.charAt(j)) {
                j = lps[j - 1];
            }
            if (combined.charAt(i) == combined.charAt(j)) {
                j++;
            }
            lps[i] = j;
        }
        int palinLen = lps[combined.length() - 1]; // ← longest palindromic prefix length
        String suffix = reversed.substring(0, s.length() - palinLen);
        return suffix + s;
    }
}
```

Time O(n) | **Space** O(n)

#228 Summary Ranges

Description: Given a sorted unique integer array, summarize consecutive runs as ranges.
Intuition: Track the start of each consecutive run; when the run breaks, emit either a single number or a "start->end" range.

Variables: `i` = scan index · `start` = index where the current run began **Pseudocode:**

```
i = 0
while i < length:
  start = i
  extend i while the next number is exactly one more
  if start equals i: emit the single number
  else: emit "nums[start]->nums[i]"
  advance i past the run
return ranges
```

```java
class Solution {
    public List<String> summaryRanges(int[] nums) {
        List<String> result = new ArrayList<>();
        int i = 0, n = nums.length;
        while (i < n) {
            int start = i;
            while (i + 1 < n && nums[i + 1] == nums[i] + 1) {  // ← extend consecutive run
                i++;
            }
            if (start == i) {
                result.add(String.valueOf(nums[start]));
            } else {
                result.add(nums[start] + "->" + nums[i]);
            }
            i++;
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#246 Strobogrammatic Number

Description: Check if a number reads the same when rotated 180 degrees.
Intuition: Two pointers move inward; each pair must be a valid strobogrammatic mapping (0-0, 1-1, 6-9, 8-8, 9-6).

Variables: `map` = digit → its 180-degree rotation · `i` / `j` = pointers moving inward · `a` / `b` = the paired chars
Pseudocode:

```
map the rotatable digits: 0->0, 1->1, 6->9, 8->8, 9->6
i = 0, j = last index
while i <= j:
  a = char at i, b = char at j
  if a is not rotatable or its rotation is not b: return false
  move i right, j left
return true
```

```java
class Solution {
    public boolean isStrobogrammatic(String num) {
        Map<Character, Character> map = new HashMap<>();
        map.put('0', '0'); map.put('1', '1'); map.put('6', '9');
        map.put('8', '8'); map.put('9', '6');
        int i = 0, j = num.length() - 1;
        while (i <= j) {
            char a = num.charAt(i), b = num.charAt(j);
            if (!map.containsKey(a) || map.get(a) != b) {   // ← invalid rotation pair
                return false;
            }
            i++;
            j--;
        }
        return true;
    }
}
```

Time O(n) | **Space** O(1)

#249 Group Shifted Strings

Description: Group strings that follow the same shift sequence (each char shifted by the same amount).
Intuition: Encode each string by the gaps between consecutive chars (mod 26) so all members of a shift group share the same key.

Variables: `map` = key → list of strings · `builder` = the gap-sequence key · `diff` = gap between consecutive chars mod 26 · `key` = finished gap string **Pseudocode:**

```
make empty map from key to list
for each string s:
  for each adjacent pair: diff = (later char − earlier char + 26) mod 26; append diff and a comma
  key = the gap sequence string
  add s to map[key]
return all the lists
```

```java
class Solution {
    public List<List<String>> groupStrings(String[] strings) {
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strings) {
            StringBuilder builder = new StringBuilder();
            for (int i = 1; i < s.length(); i++) {
                int diff = (s.charAt(i) - s.charAt(i - 1) + 26) % 26;  // ← gap mod 26
                builder.append(diff).append(',');
            }
            String key = builder.toString();
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(map.values());
    }
}
```

Time O(n·k) | **Space** O(n·k)

#266 Palindrome Permutation

Description: Check if a string can be rearranged into a palindrome (at most one odd-count char).
Intuition: A palindrome allows at most one character with an odd count; track parity with a set.

Variables: `odd` = set of chars currently seen an odd number of times **Pseudocode:**

```
make empty set odd
for each char:
  if it is in odd: remove it (now even)
  else: add it (now odd)
return true if at most one char remains odd
```

```java
class Solution {
    public boolean canPermutePalindrome(String s) {
        Set<Character> odd = new HashSet<>();
        for (char ch : s.toCharArray()) {
            if (odd.contains(ch)) {
                odd.remove(ch);                  // ← toggle parity
            } else {
                odd.add(ch);
            }
        }
        return odd.size() <= 1;                  // ← at most one odd count
    }
}
```

Time O(n) | **Space** O(1)

#271 Encode and Decode Strings

Description: Encode a list of strings to a single string and decode it back.
Intuition: Length-prefix each string ("len#str") so the decoder always knows exactly how many chars to read, regardless of content.

Variables: `builder` = encoded output · `i` = decode scan position · `j` = position of the '#' delimiter · `length` = parsed length of the next string **Pseudocode:**

```
encode: for each string append its length, then '#', then the string; return the buffer
decode:
  i = 0
  while i < length:
    scan from i to the next '#' to read the length
    parse the length, then take that many chars after '#' as one string
    advance i past that string
  return the list
```

```java
public class Codec {
    public String encode(List<String> strs) {
        StringBuilder builder = new StringBuilder();
        for (String s : strs) {
            builder.append(s.length()).append('#').append(s);  // ← length-prefixed
        }
        return builder.toString();
    }

    public List<String> decode(String s) {
        List<String> result = new ArrayList<>();
        int i = 0;
        while (i < s.length()) {
            int j = i;
            while (s.charAt(j) != '#') {           // ← read length
                j++;
            }
            int length = Integer.parseInt(s.substring(i, j));
            result.add(s.substring(j + 1, j + 1 + length));
            i = j + 1 + length;
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#273 Integer to English Words

Description: Convert a non-negative integer to its English words representation.
Intuition: Process the number in groups of three digits, naming each group and appending the right scale word (Thousand, Million, Billion).

Variables: `BELOW_20` / `TENS` / `SCALES` = word lookup tables · `scale` = which 1000-group (0=units, 1=Thousand, ...) · `group` = current 3-digit chunk · `builder` = words assembled · `threeDigits(n)` = words for a 0–999 chunk
Pseudocode:

```
if num is 0: return "Zero"
scale = 0
while num > 0:
  group = num mod 1000
  if group != 0:
    words = threeDigits(group); if scale > 0 append the scale word
    prepend words to the front of the result
  num = num / 1000; scale += 1
return trimmed result

threeDigits(n):
  if n >= 100: append hundreds word + "Hundred", drop the hundreds
  if n >= 20: append the tens word, drop the tens
  if n > 0: append the below-20 word
  return it
```

```java
class Solution {
    private static final String[] BELOW_20 = {"", "One", "Two", "Three", "Four", "Five",
        "Six", "Seven", "Eight", "Nine", "Ten", "Eleven", "Twelve", "Thirteen", "Fourteen",
        "Fifteen", "Sixteen", "Seventeen", "Eighteen", "Nineteen"};
    private static final String[] TENS = {"", "", "Twenty", "Thirty", "Forty", "Fifty",
        "Sixty", "Seventy", "Eighty", "Ninety"};
    private static final String[] SCALES = {"", "Thousand", "Million", "Billion"};

    public String numberToWords(int num) {
        if (num == 0) {
            return "Zero";
        }
        StringBuilder builder = new StringBuilder();
        int scale = 0;
        while (num > 0) {
            int group = num % 1000;
            if (group != 0) {
                String words = threeDigits(group).trim();
                if (scale > 0) {
                    words += " " + SCALES[scale];
                }
                builder.insert(0, words.trim() + " ");   // ← prepend higher groups
            }
            num /= 1000;
            scale++;
        }
        return builder.toString().trim();
    }

    private String threeDigits(int n) {
        StringBuilder builder = new StringBuilder();
        if (n >= 100) {
            builder.append(BELOW_20[n / 100]).append(" Hundred ");
            n %= 100;
        }
        if (n >= 20) {
            builder.append(TENS[n / 10]).append(' ');
            n %= 10;
        }
        if (n > 0) {
            builder.append(BELOW_20[n]).append(' ');
        }
        return builder.toString();
    }
}
```

Time O(1) | **Space** O(1)

#290 Word Pattern

Description: Check if a string `s` follows a given pattern (bijective character-to-word mapping).
Intuition: Maintain a two-way mapping between pattern chars and words; any conflict breaks the bijection.

Variables: `words` = s split on spaces · `charToWord` = pattern char → word · `wordToChar` = word → pattern char · `ch` / `word` = current pair **Pseudocode:**

```
split s into words; if count differs from pattern length: return false
make both maps empty
for each index i with ch = pattern[i], word = words[i]:
  if charToWord has ch but maps to a different word: return false
  if wordToChar has word but maps to a different char: return false
  link ch->word and word->ch
return true
```

```
class Solution {
    public boolean wordPattern(String pattern, String s) {
        String[] words = s.split(" ");
        if (pattern.length() != words.length) {
            return false;
        }
        Map<Character, String> charToWord = new HashMap<>();
        Map<String, Character> wordToChar = new HashMap<>();
        for (int i = 0; i < pattern.length(); i++) {
            char ch = pattern.charAt(i);
            String word = words[i];
            if (charToWord.containsKey(ch) && !charToWord.get(ch).equals(word)) {
                return false;
            }
            if (wordToChar.containsKey(word) && wordToChar.get(word) != ch) {
                return false;                        // ← conflict in reverse map
            }
            charToWord.put(ch, word);
            wordToChar.put(word, ch);
        }
        return true;
    }
}
```

Time O(n) | **Space** O(n)

#299 Bulls and Cows

Description: Count bulls (right digit, right place) and cows (right digit, wrong place) in a number guessing game.
Intuition: Count exact-position matches as bulls; for the rest, use a digit-count array where positive entries (secret surplus) and negative entries (guess surplus) reveal cows.

Variables: `bulls` / `cows` = counts · `count[d]` = unmatched balance for digit d (secret adds, guess subtracts) · `s` / `g` = secret/guess char at this index **Pseudocode:**

```
bulls = cows = 0; make count of size 10
for each index i with s = secret[i], g = guess[i]:
  if s equals g: bulls += 1
  else:
    if count[s] is negative (guess already had s waiting): cows += 1
    if count[g] is positive (secret already had g waiting): cows += 1
    count[s] += 1; count[g] -= 1
return "<bulls>A<cows>B"
```

```java
class Solution {
    public String getHint(String secret, String guess) {
        int bulls = 0, cows = 0;
        int[] count = new int[10];
        for (int i = 0; i < secret.length(); i++) {
            char s = secret.charAt(i), g = guess.charAt(i);
            if (s == g) {
                bulls++;
            } else {
                if (count[s - '0'] < 0) {          // ← guess earlier had this digit
                    cows++;
                }
                if (count[g - '0'] > 0) {          // ← secret earlier had this digit
                    cows++;
                }
                count[s - '0']++;
                count[g - '0']--;
            }
        }
        return bulls + "A" + cows + "B";
    }
}
```

Time O(n) | **Space** O(1)

#344 Reverse String

Description: Reverse a character array in-place.
Intuition: Two pointers swap from both ends moving inward.

Variables: `i / j` = left/right pointers · `tmp` = swap holder **Pseudocode:**

```
i = 0, j = last index
while i < j:
  swap chars at i and j
  move i right, j left
```

```java
class Solution {
    public void reverseString(char[] s) {
        int i = 0, j = s.length - 1;
        while (i < j) {
            char tmp = s[i];
            s[i] = s[j];                           // ← swap ends
            s[j] = tmp;
            i++;
            j--;
        }
    }
}
```

Time O(n) | **Space** O(1)

#345 Reverse Vowels of a String

Description: Reverse only the vowels of a string.
Intuition: Two pointers advance until each lands on a vowel, then swap those vowels.

Variables: `chars` = mutable char array · `vowels` = the vowel set string · `i` / `j` = left/right pointers · `tmp` = swap holder
Pseudocode:

```
i = 0, j = last index
while i < j:
  advance i until it lands on a vowel
  retreat j until it lands on a vowel
  swap chars at i and j
  move i right, j left
return the array as a string
```

```
class Solution {
    public String reverseVowels(String s) {
        char[] chars = s.toCharArray();
        String vowels = "aeiouAEIOU";
        int i = 0, j = chars.length - 1;
        while (i < j) {
            while (i < j && vowels.indexOf(chars[i]) < 0) {
                i++;                                   // ← seek vowel from left
            }
            while (i < j && vowels.indexOf(chars[j]) < 0) {
                j--;
            }
            char tmp = chars[i];
            chars[i] = chars[j];
            chars[j] = tmp;
            i++;
            j--;
        }
        return new String(chars);
    }
}
```

Time O(n) | **Space** O(n)

#388 Longest Absolute File Path

Description: Find the length of the longest absolute file path in a file system string.
Intuition: Tab depth gives the nesting level; keep a stack of path lengths per level and, on hitting a file (has a dot), compute the full path length.

Variables: `depthLen` = depth → cumulative length of path up to that depth · `depth` = tab count (nesting level) · `name` = the file/dir name on this line · `length` = path length including this name · `result` = best file path length **Pseudocode:**

```
depthLen[0] = 0; result = 0
for each line:
  depth = number of leading tabs
  name = the line after the tabs
  length = depthLen[depth] + length of name
  if name contains '.' (a file): result = max(result, length + depth for the slashes)
  else (a directory): depthLen[depth+1] = length (for its children)
return result
```

```java
class Solution {
    public int lengthLongestPath(String input) {
        Map<Integer, Integer> depthLen = new HashMap<>();
        depthLen.put(0, 0);
        int result = 0;
        for (String line : input.split("\n")) {
            int depth = 0;
            while (depth < line.length() && line.charAt(depth) == '\t') {  // ← count tabs
                depth++;
            }
            String name = line.substring(depth);
            int length = depthLen.get(depth) + name.length();
            if (name.contains(".")) {                // ← it is a file
                result = Math.max(result, length + depth);  // +depth for the '/' separators
            } else {
                depthLen.put(depth + 1, length);   // ← directory length for children
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#392 Is Subsequence

Description: Check if string `s` is a subsequence of string `t`.
Intuition: Walk `t` with one pointer; advance the `s` pointer each time chars match. If `s` is exhausted, it is a subsequence.

Variables: `i` = how many chars of s are matched so far · `j` = scan index into t **Pseudocode:**

```
i = 0
for j over t (stop early if all of s matched):
  if s[i] equals t[j]: advance i
return true if i reached the end of s
```

```java
class Solution {
    public boolean isSubsequence(String s, String t) {
        int i = 0;
        for (int j = 0; j < t.length() && i < s.length(); j++) {
            if (s.charAt(i) == t.charAt(j)) {
                i++;                                // ← matched next char of s
            }
        }
        return i == s.length();
    }
}
```

Time O(n) | **Space** O(1)

#408 Valid Word Abbreviation

Description: Check if an abbreviation matches a word (digits represent skipped characters).
Intuition: Walk both with pointers; on a digit, parse the skip count (no leading zero) and jump the word pointer; otherwise chars must match.

Variables: `i` = index into word · `j` = index into abbr · `ch` = current abbr char · `num` = parsed skip count **Pseudocode:**

```
i = 0, j = 0
while both have chars left:
  ch = abbr[j]
  if ch is a digit:
    if it is '0': fail (no leading zero)
    parse the full number, advancing j; jump i forward by that count
  else:
    word[i] must equal ch, else fail; advance both i and j
return true if both i and j reached their ends
```

```java
class Solution {
    public boolean validWordAbbreviation(String word, String abbr) {
        int i = 0, j = 0;
        while (i < word.length() && j < abbr.length()) {
            char ch = abbr.charAt(j);
            if (Character.isDigit(ch)) {
                if (ch == '0') {
                    return false;                  // ← no leading zero
                }
                int num = 0;
                while (j < abbr.length() && Character.isDigit(abbr.charAt(j))) {
                    num = num * 10 + (abbr.charAt(j) - '0');   // ← parse skip count
                    j++;
                }
                i += num;
            } else {
                if (word.charAt(i) != ch) {
                    return false;
                }
                i++;
                j++;
            }
        }
        return i == word.length() && j == abbr.length();
    }
}
```

Time O(n) | **Space** O(1)

#409 Longest Palindrome

Description: Find the length of the longest palindrome that can be built from the given letters.
Intuition: Use every pair of each letter; if any letter has a leftover single, one of them can sit in the center.

Variables: `count[ch]` = frequency of each char · `length` = palindrome length from full pairs · `hasOdd` = some letter has an odd count **Pseudocode:**

```
count every char
length = 0, hasOdd = false
for each count c:
  add the largest even part (c/2*2) to length
  if c is odd: hasOdd = true
return length plus 1 if any odd char can sit in the center
```

```java
class Solution {
    public int longestPalindrome(String s) {
        int[] count = new int[128];
        for (char ch : s.toCharArray()) {
            count[ch]++;
        }
        int length = 0;
        boolean hasOdd = false;
        for (int c : count) {
            length += c / 2 * 2;                    // ← use full pairs
            if (c % 2 == 1) {
                hasOdd = true;
            }
        }
        return length + (hasOdd ? 1 : 0);          // ← one odd char in the center
    }
}
```

Time O(n) | **Space** O(1)

#415 Add Strings

Description: Add two non-negative integers given as strings. Return the sum as a string.
Intuition: Add digit by digit from the right with a carry, exactly like grade-school addition.

Variables: `builder` = result digits (reversed) · `i / j` = right-to-left indices into num1/num2 · `carry` = carry into the next column · `sum` = column total **Pseudocode:**

```
i = last of num1, j = last of num2, carry = 0
while either index valid or carry remains:
  sum = carry
  add num1[i] if valid (and decrement i)
  add num2[j] if valid (and decrement j)
  append sum mod 10; carry = sum / 10
reverse the builder and return
```

```java
class Solution {
    public String addStrings(String num1, String num2) {
        StringBuilder builder = new StringBuilder();
        int i = num1.length() - 1, j = num2.length() - 1, carry = 0;
        while (i >= 0 || j >= 0 || carry > 0) {
            int sum = carry;
            if (i >= 0) {
                sum += num1.charAt(i--) - '0';
            }
            if (j >= 0) {
                sum += num2.charAt(j--) - '0';
            }
            builder.append(sum % 10);               // ← current digit
            carry = sum / 10;                       // ← carry up
        }
        return builder.reverse().toString();
    }
}
```

Time O(max(n,m)) | **Space** O(max(n,m))

#422 Valid Word Square

Description: Given a sequence of words, check whether they form a valid word square (kth row equals kth column).
Intuition: For each cell (i,j), the char must equal the char at (j,i); guard against ragged rows by bounds-checking.

Variables: `i` = row index · `j` = column index within row i **Pseudocode:**

```
for each row i:
  for each column j in that row:
    if j is out of row range, or i is past word j's length, or char (i,j) != char (j,i): return false
return true
```

```
class Solution {
    public boolean validWordSquare(List<String> words) {
        for (int i = 0; i < words.size(); i++) {
            for (int j = 0; j < words.get(i).length(); j++) {
                if (j >= words.size() || i >= words.get(j).length()
                        || words.get(i).charAt(j) != words.get(j).charAt(i)) {  // ← symmetry
                    return false;
                }
            }
        }
        return true;
    }
}
```

Time O(n·m) | **Space** O(1)

#423 Reconstruct Original Digits from English

Description: Given a scrambled string of digit-word letters, decode the original digits in order.
Intuition: Some letters are unique to one digit (z→zero, w→two, u→four, x→six, g→eight); count those first, then peel off the remaining digits using letters that become unique after subtraction.

Variables: `freq[ch-'a']` = letter frequencies · `count[d]` = how many times digit d appears · `builder` = digits in order
Pseudocode:

```
count all 26 letters
unique-letter digits first: zero(z), two(w), four(u), six(x), eight(g)
then digits whose letter is now unique after subtracting: three(h minus eight), five(f minus four), seven
then one(o minus zero,two,four) and nine(i minus five,six,eight)
for d from 0 to 9: append d, count[d] times
return the digits
```

```java
class Solution {
    public String originalDigits(String s) {
        int[] freq = new int[26];
        for (char ch : s.toCharArray()) {
            freq[ch - 'a']++;
        }
        int[] count = new int[10];
        count[0] = freq['z' - 'a'];                // zero
        count[2] = freq['w' - 'a'];                // two
        count[4] = freq['u' - 'a'];                // four
        count[6] = freq['x' - 'a'];                // six
        count[8] = freq['g' - 'a'];                // eight
        count[3] = freq['h' - 'a'] - count[8];     // three (h also in eight)
        count[5] = freq['f' - 'a'] - count[4];     // five (f also in four)
        count[7] = freq['s' - 'a'] - count[6];     // seven (s also in six)
        count[1] = freq['o' - 'a'] - count[0] - count[2] - count[4];  // one
        count[9] = freq['i' - 'a'] - count[5] - count[6] - count[8];  // nine
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < 10; i++) {
            for (int j = 0; j < count[i]; j++) {
                builder.append(i);
            }
        }
        return builder.toString();
    }
}
```

Time O(n) | **Space** O(1)

#434 Number of Segments in a String

Description: Count the number of segments (maximal runs of non-space chars) in a string.
Intuition: A new segment starts at each non-space char whose predecessor is a space (or string start).

Variables: `count` = number of segments · `i` = scan index **Pseudocode:**

```
count = 0
for each index i:
  if char is not a space and (i is 0 or previous char is a space): count += 1 (new segment starts)
return count
```

```java
class Solution {
    public int countSegments(String s) {
        int count = 0;
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) != ' ' && (i == 0 || s.charAt(i - 1) == ' ')) {  // ← segment start
                count++;
            }
        }
        return count;
    }
}
```

Time O(n) | **Space** O(1)

#459 Repeated Substring Pattern

Description: Check if a string can be built by repeating one of its substrings multiple times.
Intuition: If `s` is a repeated pattern, it appears inside `(s+s)` with the first and last char removed.

Variables: `doubled` = (s+s) with the first and last char stripped **Pseudocode:**

```
doubled = s+s with the first and last character removed
return whether doubled contains s
```

```
class Solution {
    public boolean repeatedSubstringPattern(String s) {
        String doubled = (s + s).substring(1, 2 * s.length() - 1);  // ← strip first+last char
        return doubled.contains(s);
    }
}
```

Time O(n) | **Space** O(n)

#468 Validate IP Address

Description: Validate whether a string is a valid IPv4 or IPv6 address.
Intuition: Dispatch on the separator: dots mean IPv4 (four 0–255 octets, no leading zeros), colons mean IPv6 (eight 1–4 hex groups).

Variables: `parts` = the address split on its separator · `part` = one octet/group · `isIPv4` / `isIPv6` = validators
Pseudocode:

```
if it contains '.': return "IPv4" if isIPv4 else "Neither"
if it contains ':': return "IPv6" if isIPv6 else "Neither"
otherwise: "Neither"

isIPv4: split on '.'; need exactly 4 parts; each part: non-empty, <=3 chars, all digits, no leading zero,
isIPv6: split on ':'; need exactly 8 parts; each part: non-empty, <=4 chars, all valid hex digits
```

```java
class Solution {
    public String validIPAddress(String queryIP) {
        if (queryIP.contains(".")) {
            return isIPv4(queryIP) ? "IPv4" : "Neither";
        } else if (queryIP.contains(":")) {
            return isIPv6(queryIP) ? "IPv6" : "Neither";
        }
        return "Neither";
    }

    private boolean isIPv4(String s) {
        String[] parts = s.split("\\.", -1);
        if (parts.length != 4) {
            return false;
        }
        for (String part : parts) {
            if (part.isEmpty() || part.length() > 3) {
                return false;
            }
            for (char ch : part.toCharArray()) {
                if (!Character.isDigit(ch)) {
                    return false;
                }
            }
            if (part.length() > 1 && part.charAt(0) == '0') {   // ← no leading zero
                return false;
            }
            if (Integer.parseInt(part) > 255) {
                return false;
            }
        }
        return true;
    }

    private boolean isIPv6(String s) {
        String[] parts = s.split(":", -1);
        if (parts.length != 8) {
            return false;
        }
        for (String part : parts) {
            if (part.isEmpty() || part.length() > 4) {
                return false;
            }
            for (char ch : part.toCharArray()) {
                if (Character.digit(ch, 16) < 0) {  // ← valid hex digit
                    return false;
                }
            }
        }
        return true;
    }
}
```

Time O(n) | **Space** O(n)

#500 Keyboard Row

Description: Find all words that can be typed on a single row of a QWERTY keyboard.
Intuition: Map each letter to its row index, then keep words whose letters all share one row.

Variables: `rows` = the three keyboard rows · `rowOf[ch-'a']` = which row each letter is on · `row` = the first letter's row · `ok` = all letters share that row **Pseudocode:**

```
fill rowOf so each letter knows its keyboard row
for each word:
  row = row of its first letter (lowercased)
  ok = true
  for each letter (lowercased): if its row differs, ok = false, stop
  if ok: keep the word
return the kept words
```

```java
class Solution {
    public String[] findWords(String[] words) {
        String[] rows = {"qwertyuiop", "asdfghjkl", "zxcvbnm"};
        int[] rowOf = new int[26];
        for (int r = 0; r < rows.length; r++) {
            for (char ch : rows[r].toCharArray()) {
                rowOf[ch - 'a'] = r;
            }
        }
        List<String> result = new ArrayList<>();
        for (String word : words) {
            int row = rowOf[Character.toLowerCase(word.charAt(0)) - 'a'];
            boolean ok = true;
            for (char ch : word.toLowerCase().toCharArray()) {
                if (rowOf[ch - 'a'] != row) {      // ← differs from first letter's row
                    ok = false;
                    break;
                }
            }
            if (ok) {
                result.add(word);
            }
        }
        return result.toArray(new String[0]);
    }
}
```

Time O(n·k) | **Space** O(n)

#520 Detect Capital

Description: Check if a word is capitalized correctly (all caps, all lower, or first letter only).
Intuition: Count uppercase letters: valid only if all are uppercase, none are, or exactly the first one is.

Variables: `upper` = number of uppercase letters in the word **Pseudocode:**

```
upper = count of uppercase letters
if upper equals the length (all caps) or upper is 0 (all lower): return true
return true only if upper is exactly 1 and the first letter is uppercase
```

```java
class Solution {
    public boolean detectCapitalUse(String word) {
        int upper = 0;
        for (char ch : word.toCharArray()) {
            if (Character.isUpperCase(ch)) {
                upper++;
            }
        }
        if (upper == word.length() || upper == 0) {
            return true;                              // ← all caps or all lower
        }
        return upper == 1 && Character.isUpperCase(word.charAt(0));  // ← only first
    }
}
```

Time O(n) | **Space** O(1)

#524 Longest Word in Dictionary through Deleting

Description: Find the longest word in a dictionary that is a subsequence of string `s` (smallest lexicographically on ties). **Intuition:** Check each candidate with the subsequence two-pointer, keeping the best by length then lexicographic order.

Variables: `result` = best word found so far · `isSubsequence(word,s)` = two-pointer subsequence check · `i` = matched chars of word **Pseudocode:**

```
result = ""
for each word in the dictionary:
  if word is a subsequence of s:
    if it is longer, or same length but lexicographically smaller: result = word
return result

isSubsequence(word, s): walk s; advance a word pointer on each match; true if all of word matched
```

```java
class Solution {
    public String findLongestWord(String s, List<String> dictionary) {
        String result = "";
        for (String word : dictionary) {
            if (isSubsequence(word, s)) {
                if (word.length() > result.length()
                        || (word.length() == result.length() && word.compareTo(result) < 0)) {
                    result = word;                 // ← longer, or same length but smaller
                }
            }
        }
        return result;
    }
    private boolean isSubsequence(String word, String s) {
        int i = 0;
        for (int j = 0; j < s.length() && i < word.length(); j++) {
            if (word.charAt(i) == s.charAt(j)) {
                i++;
            }
        }
        return i == word.length();
    }
}
```

Time O(n·k) | **Space** O(1)

#541 Reverse String II

Description: Reverse the first `k` characters of every `2k`-character block in a string.
Intuition: Step through the array in `2k` jumps, reversing the first `k` chars of each block (or all that remain).

Variables: `chars` = mutable array · `i` = start of each 2k block · `left`/`right` = bounds of the first k chars to reverse · `tmp` = swap holder **Pseudocode:**

```
for i stepping by 2k through the array:
  left = i, right = min(i+k-1, last index) (first k of this block)
  swap inward from left/right until they meet
return the array as a string
```

```
class Solution {
    public String reverseStr(String s, int k) {
        char[] chars = s.toCharArray();
        for (int i = 0; i < chars.length; i += 2 * k) {
            int left = i, right = Math.min(i + k - 1, chars.length - 1);  // ← first k of block
            while (left < right) {
                char tmp = chars[left];
                chars[left] = chars[right];
                chars[right] = tmp;
                left++;
                right--;
            }
        }
        return new String(chars);
    }
}
```

Time O(n) | **Space** O(n)

#551 Student Attendance Record I

Description: Check if a record is award-eligible (fewer than 2 absences total, no 3 consecutive lates).
Intuition: Count total absences and track the current run of lates; fail on the second absence or third consecutive late.

Variables: `absent` = total absences · `lateStreak` = current run of consecutive lates **Pseudocode:**

```
absent = 0, lateStreak = 0
for each char:
  if 'A': absent += 1; fail if it reaches 2
  if 'L': lateStreak += 1; fail if it reaches 3
  else: reset lateStreak to 0
return true
```

```java
class Solution {
    public boolean checkRecord(String s) {
        int absent = 0, lateStreak = 0;
        for (char ch : s.toCharArray()) {
            if (ch == 'A') {
                absent++;
                if (absent >= 2) {
                    return false;
                }
            }
            if (ch == 'L') {
                lateStreak++;
                if (lateStreak >= 3) {             // ← 3 consecutive lates
                    return false;
                }
            } else {
                lateStreak = 0;                    // ← reset streak
            }
        }
        return true;
    }
}
```

Time O(n) | **Space** O(1)

#556 Next Greater Element III

Description: Find the next greater number by rearranging the digits of `n`. Return -1 if none or on overflow.
Intuition: Standard next-permutation on digits: find the rightmost ascending pair, swap with the next larger digit to its right, reverse the suffix.

Variables: `digits` = digits of n as a char array · `i` = pivot (rightmost digit smaller than its right neighbor) · `j` = next larger digit to the right · `value` = parsed result (long, for overflow) **Pseudocode:**

```
i = second-to-last; move left while digits[i] >= digits[i+1] (find the pivot)
if no pivot (i < 0): return -1
j = last; move left while digits[j] <= digits[i] (next larger digit)
swap digits at i and j
reverse the suffix after i (smallest arrangement)
parse as a long; return -1 if it overflows int, else the int
```

```java
class Solution {
    public int nextGreaterElement(int n) {
        char[] digits = String.valueOf(n).toCharArray();
        int i = digits.length - 2;
        while (i >= 0 && digits[i] >= digits[i + 1]) {  // ← find pivot
            i--;
        }
        if (i < 0) {
            return -1;
        }
        int j = digits.length - 1;
        while (digits[j] <= digits[i]) {           // ← next larger to the right
            j--;
        }
        swap(digits, i, j);
        reverse(digits, i + 1, digits.length - 1); // ← smallest suffix
        long value = Long.parseLong(new String(digits));
        return value > Integer.MAX_VALUE ? -1 : (int) value;
    }
    private void swap(char[] a, int i, int j) {
        char tmp = a[i];
        a[i] = a[j];
        a[j] = tmp;
    }
    private void reverse(char[] a, int i, int j) {
        while (i < j) {
            swap(a, i++, j--);
        }
    }
}
```

Time O(d) | **Space** O(d)

#557 Reverse Words in a String III

Description: Reverse individual words in a string while preserving word order.
Intuition: Split on spaces, reverse each word, and rejoin with single spaces.

Variables: `words` = s split on spaces · `builder` = result · `i` = word index **Pseudocode:**

```
split s into words
for each word index i:
  if not the first word: append a space
  append the reversed word
return the result
```

```java
class Solution {
    public String reverseWords(String s) {
        String[] words = s.split(" ");
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < words.length; i++) {
            if (i > 0) {
                builder.append(' ');
            }
            builder.append(new StringBuilder(words[i]).reverse());  // ← reverse each word
        }
        return builder.toString();
    }
}
```

Time O(n) | **Space** O(n)

#592 Fraction Addition and Subtraction

Description: Add or subtract a sequence of fractions, returning the result in lowest terms.
Intuition: Parse each signed fraction, accumulate over a common denominator, then reduce by the GCD.

Variables: `num` / `den` = running result fraction · `i` = scan index · `sign` = +1/-1 of the current fraction · `a` / `b` = current numerator/denominator · `g` = GCD for reducing **Pseudocode:**

```
num = 0, den = 1, i = 0
while i < length:
  read an optional sign
  parse numerator a (digits)
  skip the '/'
  parse denominator b (digits)
  num = num*b + sign*a*den; den = den*b (add over a common denominator)
g = gcd(|num|, den)
return (num/g) + "/" + (den/g)
```

```
class Solution {
    public String fractionAddition(String expression) {
        int num = 0, den = 1;
        int i = 0, n = expression.length();
        while (i < n) {
            int sign = 1;
            if (expression.charAt(i) == '+' || expression.charAt(i) == '-') {
                sign = expression.charAt(i) == '-' ? -1 : 1;
                i++;
            }
            int a = 0;
            while (i < n && Character.isDigit(expression.charAt(i))) {
                a = a * 10 + (expression.charAt(i) - '0');   // ← numerator
                i++;
            }
            i++;                                   // skip '/'
            int b = 0;
            while (i < n && Character.isDigit(expression.charAt(i))) {
                b = b * 10 + (expression.charAt(i) - '0');   // ← denominator
                i++;
            }
            num = num * b + sign * a * den;        // ← add over common denominator
            den *= b;
        }
        int g = gcd(Math.abs(num), den);
        return (num / g) + "/" + (den / g);
    }
    private int gcd(int a, int b) {
        return b == 0 ? Math.max(a, 1) : gcd(b, a % b);
    }
}
```

Time O(n) | **Space** O(1)

#616 Add Bold Tag in String

Description: Wrap every substring of `s` that matches a word in the dictionary with `<b>...</b>`, merging overlaps.
Intuition: Mark every covered index in a boolean array, then emit bold tags around maximal marked runs.

Variables: `bold[i]` = whether index i is covered by some word · `start` = each match position · `builder` = tagged result **Pseudocode:**

```
make bold array all false
for each dictionary word:
  find every occurrence in s; mark all its indices bold
build the result:
  for each index i:
    if bold[i] and (i is 0 or previous not bold): open "<b>"
    append the char
    if bold[i] and (i is last or next not bold): close "</b>"
return the result
```

```
class Solution {
    public String addBoldTag(String s, String[] words) {
        boolean[] bold = new boolean[s.length()];
        for (String word : words) {
            int start = s.indexOf(word);
            while (start >= 0) {
                for (int i = start; i < start + word.length(); i++) {
                    bold[i] = true;                // ← mark covered indices
                }
                start = s.indexOf(word, start + 1);
            }
        }
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < s.length(); i++) {
            if (bold[i] && (i == 0 || !bold[i - 1])) {
                builder.append("<b>");             // ← open at run start
            }
            builder.append(s.charAt(i));
            if (bold[i] && (i == s.length() - 1 || !bold[i + 1])) {
                builder.append("</b>");            // ← close at run end
            }
        }
        return builder.toString();
    }
}
```

Time O(n·m) | **Space** O(n)

#657 Robot Return to Origin

Description: Check if a robot following an instruction string (UDLR) returns to the origin.
Intuition: Track net horizontal and vertical displacement; the robot returns only if both are zero.

Variables: `x / y` = net horizontal/vertical displacement **Pseudocode:**

```
x = 0, y = 0
for each move: U adds to y, D subtracts from y, L subtracts from x, R adds to x
return true if both x and y are 0
```

```java
class Solution {
    public boolean judgeCircle(String moves) {
        int x = 0, y = 0;
        for (char ch : moves.toCharArray()) {
            if (ch == 'U') {
                y++;
            } else if (ch == 'D') {
                y--;
            } else if (ch == 'L') {
                x--;
            } else {
                x++;
            }
        }
        return x == 0 && y == 0;                    // ← back at origin
    }
}
```

Time O(n) | **Space** O(1)

#670 Maximum Swap

Description: Swap two digits at most once to get the maximum possible value.
Intuition: Record the last index of each digit; scanning left to right, swap the first digit with the largest later digit that exceeds it.

Variables: `digits` = digits of num as a char array · `last[d]` = last index where digit d appears · `i` = scan index · `d` = a candidate larger digit **Pseudocode:**

```
record the last position of each digit 0–9
for each index i left to right:
  for d from 9 down to (digits[i]+1):
    if d appears after i: swap digits[i] with that later d, return the new number
return num unchanged
```

```java
class Solution {
    public int maximumSwap(int num) {
        char[] digits = String.valueOf(num).toCharArray();
        int[] last = new int[10];
        for (int i = 0; i < digits.length; i++) {
            last[digits[i] - '0'] = i;              // ← last position of each digit
        }
        for (int i = 0; i < digits.length; i++) {
            for (int d = 9; d > digits[i] - '0'; d--) {
                if (last[d] > i) {                  // ← a larger digit appears later
                    char tmp = digits[i];
                    digits[i] = digits[last[d]];
                    digits[last[d]] = tmp;
                    return Integer.parseInt(new String(digits));
                }
            }
        }
        return num;
    }
}
```

Time O(d) | **Space** O(1)

#680 Valid Palindrome II

Description: Check if a string is a palindrome after deleting at most one character.
Intuition: Two pointers inward; at the first mismatch, try skipping either the left or right char and check the remainder.

Variables: `i / j` = pointers moving inward · `isPalindrome(i,j)` = plain palindrome check on a range **Pseudocode:**

```
i = 0, j = last index
while i < j:
  if chars at i and j differ: return true if skipping the left char OR the right char leaves a palindrome
  move i right, j left
return true

isPalindrome(i, j): move inward; false on any mismatch
```

```
class Solution {
    public boolean validPalindrome(String s) {
        int i = 0, j = s.length() - 1;
        while (i < j) {
            if (s.charAt(i) != s.charAt(j)) {
                return isPalindrome(s, i + 1, j) || isPalindrome(s, i, j - 1);  // ← skip one
            }
            i++;
            j--;
        }
        return true;
    }
    private boolean isPalindrome(String s, int i, int j) {
        while (i < j) {
            if (s.charAt(i) != s.charAt(j)) {
                return false;
            }
            i++;
            j--;
        }
        return true;
    }
}
```

Time O(n) | **Space** O(1)

#681 Next Closest Time

Description: Find the next closest valid time using only the digits present in a given time string.
Intuition: From the current minute, advance one minute at a time and return the first time whose digits are all in the allowed set.

Variables: `allowed` = set of digit chars present in the input · `minutes` = current time in minutes since midnight · `candidate` = formatted next time · `ok` = all its digits allowed **Pseudocode:**

```
collect the allowed digit chars (skip ':')
minutes = hours*60 + minutes of the input
loop:
  minutes = (minutes + 1) mod (24*60) (advance one minute)
  format as HH:MM
  if every digit of it is allowed: return it
```

```java
class Solution {
    public String nextClosestTime(String time) {
        Set<Character> allowed = new HashSet<>();
        for (char ch : time.toCharArray()) {
            if (ch != ':') {
                allowed.add(ch);
            }
        }
        int minutes = Integer.parseInt(time.substring(0, 2)) * 60
                    + Integer.parseInt(time.substring(3));
        while (true) {
            minutes = (minutes + 1) % (24 * 60);   // ← advance one minute
            String candidate = String.format("%02d:%02d", minutes / 60, minutes % 60);
            boolean ok = true;
            for (char ch : candidate.toCharArray()) {
                if (ch != ':' && !allowed.contains(ch)) {
                    ok = false;
                    break;
                }
            }
            if (ok) {
                return candidate;
            }
        }
    }
}
```

Time O(1) | **Space** O(1)

#686 Repeated String Match

Description: Find the minimum number of times string `a` must repeat so `b` is a substring. Return -1 if impossible.
Intuition: Repeat `a` until its length covers `b`, then check one extra copy to handle offset overlap.

Variables: `builder` = repeated copies of a · `count` = number of copies so far **Pseudocode:**

```
repeat a (counting copies) until the buffer is at least as long as b
if the buffer contains b: return count
append one more copy of a (for offset overlap)
if it now contains b: return count + 1
return -1
```

```java
class Solution {
    public int repeatedStringMatch(String a, String b) {
        StringBuilder builder = new StringBuilder();
        int count = 0;
        while (builder.length() < b.length()) {    // ← grow until long enough
            builder.append(a);
            count++;
        }
        if (builder.indexOf(b) >= 0) {
            return count;
        }
        builder.append(a);                         // ← one more for offset overlap
        if (builder.indexOf(b) >= 0) {
            return count + 1;
        }
        return -1;
    }
}
```

Time O(n·m) | **Space** O(n)

#709 To Lower Case

Description: Convert a string to lowercase.
Intuition: Uppercase ASCII letters differ from lowercase by 32, so shift any A–Z char.

Variables: `chars` = mutable char array · `i` = scan index **Pseudocode:**

```
turn s into a char array
for each char: if it is A–Z, add 32 to shift it to lowercase
return the array as a string
```

```java
class Solution {
    public String toLowerCase(String s) {
        char[] chars = s.toCharArray();
        for (int i = 0; i < chars.length; i++) {
            if (chars[i] >= 'A' && chars[i] <= 'Z') {
                chars[i] = (char) (chars[i] + 32);  // ← shift to lowercase
            }
        }
        return new String(chars);
    }
}
```

Time O(n) | **Space** O(n)

#726 Number of Atoms

Description: Parse a chemical formula string and return atom counts in sorted order.
Intuition: Recursive-descent style parse with a stack of count maps for parentheses; multiply a group's counts by the number following its closing paren.

Variables: `i` = global parse position · `formula` = the string · `counts` = atom → count for the current scope · `parse()` = parse one group · `parseName()` = read an element name · `parseNumber()` = read a count (default 1) · `mult` = multiplier after a ')' **Pseudocode:**

```
countOfAtoms: parse the whole formula, sort atoms (TreeMap), build name + count (omit count 1)

parse():
  make empty counts
  while not at end and not at ')':
    if '(': skip '(', recurse into parse(), skip ')', read the multiplier, add inner counts times the mul
    else: read an element name, read its number, add to counts
  return counts

parseName(): take the uppercase letter then following lowercase letters
parseNumber(): read digits into a number; return 1 if there were none
```

```java
class Solution {
    private int i = 0;
    private String formula;

    public String countOfAtoms(String formula) {
        this.formula = formula;
        Map<String, Integer> counts = parse();
        TreeMap<String, Integer> sorted = new TreeMap<>(counts);
        StringBuilder builder = new StringBuilder();
        for (Map.Entry<String, Integer> e : sorted.entrySet()) {
            builder.append(e.getKey());
            if (e.getValue() > 1) {
                builder.append(e.getValue());
            }
        }
        return builder.toString();
    }

    private Map<String, Integer> parse() {
        Map<String, Integer> counts = new HashMap<>();
        while (i < formula.length() && formula.charAt(i) != ')') {
            if (formula.charAt(i) == '(') {
                i++;                                // skip '('
                Map<String, Integer> inner = parse();
                i++;                                // skip ')'
                int mult = parseNumber();
                for (Map.Entry<String, Integer> e : inner.entrySet()) {
                    counts.merge(e.getKey(), e.getValue() * mult, Integer::sum);  // ← scale group
                }
            } else {
                String name = parseName();
                int num = parseNumber();
                counts.merge(name, num, Integer::sum);
            }
        }
        return counts;
    }

    private String parseName() {
        int start = i++;
        while (i < formula.length() && Character.isLowerCase(formula.charAt(i))) {
            i++;
        }
        return formula.substring(start, i);
    }

    private int parseNumber() {
        int num = 0;
        while (i < formula.length() && Character.isDigit(formula.charAt(i))) {
            num = num * 10 + (formula.charAt(i) - '0');
            i++;
        }
        return num == 0 ? 1 : num;                  // ← implicit count of 1
    }
}
```

Time O(n) | **Space** O(n)

#748 Shortest Completing Word

Description: Find the shortest word in a list that contains all letters of a license plate (case-insensitive, with multiplicity).
Intuition: Build the plate's letter-count vector, then keep the shortest word whose own counts cover it.

Variables: `target[ch-'a']` = required count of each letter from the plate · `count[ch-'a']` = a word's letter counts · `covers` = word has enough of every letter · `result` = shortest covering word so far **Pseudocode:**

```
build target counts from the plate's letters (lowercased, letters only)
result = none
for each word:
  count its letters
  covers = true; if any letter's count is below target, covers = false
  if covers and (no result yet or this word is shorter): result = word
return result
```

```java
class Solution {
    public String shortestCompletingWord(String licensePlate, String[] words) {
        int[] target = new int[26];
        for (char ch : licensePlate.toLowerCase().toCharArray()) {
            if (Character.isLetter(ch)) {
                target[ch - 'a']++;
            }
        }
        String result = null;
        for (String word : words) {
            int[] count = new int[26];
            for (char ch : word.toCharArray()) {
                count[ch - 'a']++;
            }
            boolean covers = true;
            for (int i = 0; i < 26; i++) {
                if (count[i] < target[i]) {        // ← missing a required letter
                    covers = false;
                    break;
                }
            }
            if (covers && (result == null || word.length() < result.length())) {
                result = word;
            }
        }
        return result;
    }
}
```

Time O(n·k) | **Space** O(1)

#788 Rotated Digits

Description: Count numbers in `[1, n]` that become a different valid number when each digit is rotated 180 degrees.
Intuition: A number is "good" if all digits are valid rotations and at least one digit (2,5,6,9) actually changes.

Variables: `count` = how many good numbers · `isGood(num)` = valid + actually changed · `d` = a digit · `changed` = some digit flips to a different one **Pseudocode:**

```
count = 0
for i from 1 to n: if isGood(i): count += 1
return count

isGood(num):
  changed = false
  for each digit d of num:
    if d is 3, 4, or 7: invalid rotation, return false
    if d is 2, 5, 6, or 9: changed = true
  return changed
```

```java
class Solution {
    public int rotatedDigits(int n) {
        int count = 0;
        for (int i = 1; i <= n; i++) {
            if (isGood(i)) {
                count++;
            }
        }
        return count;
    }
    private boolean isGood(int num) {
        boolean changed = false;
        while (num > 0) {
            int d = num % 10;
            if (d == 3 || d == 4 || d == 7) {
                return false;                 // ← invalid rotation
            }
            if (d == 2 || d == 5 || d == 6 || d == 9) {
                changed = true;               // ← digit changes
            }
            num /= 10;
        }
        return changed;
    }
}
```

Time O(n·d) | **Space** O(1)

#791 Custom Sort String

Description: Sort string `s` so its characters appear in the order given by string `order`.
Intuition: Count chars in `s`, emit them in `order`'s sequence, then append any chars not mentioned in `order`.

Variables: `count[ch-'a']` = remaining copies of each char in s · `builder` = result **Pseudocode:**

```
count every char of s
for each char in order: append it count[ch] times and zero that count
for each remaining letter still counted: append it count times
return the result
```

```java
class Solution {
    public String customSortString(String order, String s) {
        int[] count = new int[26];
        for (char ch : s.toCharArray()) {
            count[ch - 'a']++;
        }
        StringBuilder builder = new StringBuilder();
        for (char ch : order.toCharArray()) {
            while (count[ch - 'a'] > 0) {          // ← emit in custom order
                builder.append(ch);
                count[ch - 'a']--;
            }
        }
        for (int i = 0; i < 26; i++) {
            while (count[i] > 0) {                   // ← leftover chars
                builder.append((char) ('a' + i));
                count[i]--;
            }
        }
        return builder.toString();
    }
}
```

Time O(n) | **Space** O(n)

#794 Valid Tic-Tac-Toe State

Description: Decide whether the given tic-tac-toe board is reachable from valid play.
Intuition: Count X and O; X count must equal O or be one more, and if a player has won the move counts must be consistent (both can't win).

Variables: `xCount` / `oCount` = piece counts · `xWin` / `oWin` = whether each player has three in a row · `wins(board,p)` = three-in-a-row check for player p **Pseudocode:**

```
count X and O across the board
if oCount is not equal to xCount and not xCount-1: invalid (turn order)
xWin = X has a line, oWin = O has a line
if both win: invalid
if X wins but xCount != oCount+1: invalid (X must have just moved)
if O wins but xCount != oCount: invalid (O must have just moved)
otherwise valid

wins(board, p): check all rows, columns, and both diagonals for three p's
```

```java
class Solution {
    public boolean validTicTacToe(String[] board) {
        int xCount = 0, oCount = 0;
        for (String row : board) {
            for (char ch : row.toCharArray()) {
                if (ch == 'X') {
                    xCount++;
                } else if (ch == 'O') {
                    oCount++;
                }
            }
        }
        if (oCount != xCount && oCount != xCount - 1) {
            return false;                         // ← turn order broken
        }
        boolean xWin = wins(board, 'X'), oWin = wins(board, 'O');
        if (xWin && oWin) {
            return false;                         // ← both can't win
        }
        if (xWin && xCount != oCount + 1) {
            return false;                         // ← X wins on its own move
        }
        if (oWin && xCount != oCount) {
            return false;                         // ← O wins on its own move
        }
        return true;
    }
    private boolean wins(String[] b, char p) {
        for (int i = 0; i < 3; i++) {
            if (b[i].charAt(0) == p && b[i].charAt(1) == p && b[i].charAt(2) == p) {
                return true;
            }
            if (b[0].charAt(i) == p && b[1].charAt(i) == p && b[2].charAt(i) == p) {
                return true;
            }
        }
        if (b[0].charAt(0) == p && b[1].charAt(1) == p && b[2].charAt(2) == p) {
            return true;
        }
        if (b[0].charAt(2) == p && b[1].charAt(1) == p && b[2].charAt(0) == p) {
            return true;
        }
        return false;
    }
}
```

Time O(1) | **Space** O(1)

#796 Rotate String

Description: Check if string `t` is a rotation of string `s`.
Intuition: Every rotation of `s` is a substring of `s+s`, so check containment after a length match.

Variables: `s+s` = s concatenated with itself (contains every rotation) **Pseudocode:**

```
return true if s and goal have the same length and (s+s) contains goal
```

```java
class Solution {
    public boolean rotateString(String s, String goal) {
        return s.length() == goal.length() && (s + s).contains(goal);  // ← rotation ⊂ s+s
    }
}
```

Time $O(n^2)$ | **Space** $O(n)$

#809 Expressive Words

Description: Determine how many query words match a stretched version of `s` (groups stretched to length ≥ 3).

Intuition: Compare run lengths group by group: the stretched group must be ≥ 3, or equal to the original run length.

Variables: `count` = matching words · `stretchy(s,word)` = group-by-group match · `i` / `j` = positions in s/word · `lenS` / `lenW` = run lengths at each position · `runLength(s,i)` = length of the run starting at i **Pseudocode:**

```
count = 0
for each word: if stretchy(s, word): count += 1
return count

stretchy(s, word):
  walk both with i, j:
    if the chars differ: return false
    lenS = run length in s, lenW = run length in word
    if lenS < lenW, or (they differ and lenS < 3): return false (can't stretch to match)
    advance i by lenS, j by lenW
  return true only if both fully consumed

runLength(s, i): count chars equal to s[i] starting at i
```

```java
class Solution {
    public int expressiveWords(String s, String[] words) {
        int count = 0;
        for (String word : words) {
            if (stretchy(s, word)) {
                count++;
            }
        }
        return count;
    }
    private boolean stretchy(String s, String word) {
        int i = 0, j = 0;
        while (i < s.length() && j < word.length()) {
            if (s.charAt(i) != word.charAt(j)) {
                return false;
            }
            int lenS = runLength(s, i), lenW = runLength(word, j);
            if (lenS < lenW || (lenS != lenW && lenS < 3)) {  // ← can't stretch to match
                return false;
            }
            i += lenS;
            j += lenW;
        }
        return i == s.length() && j == word.length();
    }
    private int runLength(String s, int i) {
        int j = i;
        while (j < s.length() && s.charAt(j) == s.charAt(i)) {
            j++;
        }
        return j - i;
    }
}
```

Time O(n·k) | **Space** O(1)

#824 Goat Latin

Description: Convert words to Goat Latin: append "ma" (plus per-word index of 'a's); move leading consonants to the end for non-vowel words.
Intuition: For each word, decide vowel vs consonant start, transform, then append "ma" and i+1 trailing 'a's.

Variables: `vowels` = vowel set · `words` = the sentence split · `i` = word index · `builder` = one transformed word · `result` = full output **Pseudocode:**

```
make the vowel set; split the sentence into words
for each word index i:
  if it starts with a vowel: keep it
  else: move the first letter to the end
  append "ma"
  append i+1 trailing 'a's
  separate words with single spaces
return the result
```

```java
class Solution {
    public String toGoatLatin(String sentence) {
        Set<Character> vowels = new HashSet<>(Arrays.asList('a', 'e', 'i', 'o', 'u',
            'A', 'E', 'I', 'O', 'U'));
        String[] words = sentence.split(" ");
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < words.length; i++) {
            String word = words[i];
            StringBuilder builder = new StringBuilder();
            if (vowels.contains(word.charAt(0))) {
                builder.append(word);
            } else {
                builder.append(word.substring(1)).append(word.charAt(0));  // ← move consonant
            }
            builder.append("ma");
            for (int j = 0; j <= i; j++) {
                builder.append('a');                // ← i+1 trailing a's
            }
            if (i > 0) {
                result.append(' ');
            }
            result.append(builder);
        }
        return result.toString();
    }
}
```

Time O(n) | **Space** O(n)

#833 Find And Replace in String

Description: Apply non-overlapping indexed replacements: at each index, if `sources[k]` matches, replace it with `targets[k]`.

Intuition: Map each start index to its replacement; rebuild the string, jumping over matched sources and copying unmatched chars verbatim.

Variables: `indexToOp` = start index → operation number k (only for matching sources) · `i` = rebuild scan position · `k` = the operation at index i · `builder` = result **Pseudocode:**

```
for each operation k: if sources[k] matches s at indices[k], record indices[k] -> k
i = 0
while i < length of s:
  if i has a recorded op k: append targets[k], jump i past sources[k]
  else: copy s[i], advance i
return the result
```

```
class Solution {
    public String findReplaceString(String s, int[] indices, String[] sources, String[] targets) {
        Map<Integer, Integer> indexToOp = new HashMap<>();
        for (int k = 0; k < indices.length; k++) {
            if (s.startsWith(sources[k], indices[k])) {   // ← source matches at index
                indexToOp.put(indices[k], k);
            }
        }
        StringBuilder builder = new StringBuilder();
        int i = 0;
        while (i < s.length()) {
            if (indexToOp.containsKey(i)) {
                int k = indexToOp.get(i);
                builder.append(targets[k]);        // ← apply replacement
                i += sources[k].length();
            } else {
                builder.append(s.charAt(i));
                i++;
            }
        }
        return builder.toString();
    }
}
```

Time O(n) | **Space** O(n)

#859 Buddy Strings

Description: Check if two strings are "buddy strings": swapping exactly one pair of chars makes them equal.
Intuition: Equal length is required; if the strings are identical, you need a duplicate char to swap; otherwise exactly two positions must differ and be mirror images.

Variables: `count[ch-'a']` = letter counts (for the equal case) · `first` / `second` = the two differing positions
Pseudocode:

```
if lengths differ: return false
if s equals goal:
  return true only if some letter appears twice (a swap of equals)
find the differing positions; record first then second; more than two -> false
return true only if there are exactly two and they are mirror images (s[first]=goal[second] and s[second]
```

```java
class Solution {
    public boolean buddyStrings(String s, String goal) {
        if (s.length() != goal.length()) {
            return false;
        }
        if (s.equals(goal)) {
            int[] count = new int[26];
            for (char ch : s.toCharArray()) {
                if (++count[ch - 'a'] > 1) {       // ← duplicate exists to swap
                    return true;
                }
            }
            return false;
        }
        int first = -1, second = -1;
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) != goal.charAt(i)) {
                if (first == -1) {
                    first = i;
                } else if (second == -1) {
                    second = i;
                } else {
                    return false;                  // ← more than two diffs
                }
            }
        }
        return second != -1 && s.charAt(first) == goal.charAt(second)
            && s.charAt(second) == goal.charAt(first);
    }
}
```

Time O(n) | **Space** O(1)

#953 Verifying an Alien Dictionary

Description: Check if words are sorted in a given alien language's alphabetical order.
Intuition: Map each letter to its rank, then verify each adjacent pair is in non-decreasing order under that ranking.

Variables: `rank[ch-'a']` = position of each letter in the alien order · `inOrder(a,b,rank)` = whether a comes before-or-equal b · `ra` / `rb` = ranks of the compared chars **Pseudocode:**

```
build rank from the alien order
for each adjacent pair of words: if not inOrder, return false
return true

inOrder(a, b):
  compare char by char up to the shorter length:
    if ranks differ: a is in order iff its rank is smaller
  if all equal: a must be no longer than b (prefix comes first)
```

```java
class Solution {
    public boolean isAlienSorted(String[] words, String order) {
        int[] rank = new int[26];
        for (int i = 0; i < order.length(); i++) {
            rank[order.charAt(i) - 'a'] = i;       // ← letter rank
        }
        for (int i = 0; i + 1 < words.length; i++) {
            if (!inOrder(words[i], words[i + 1], rank)) {
                return false;
            }
        }
        return true;
    }
    private boolean inOrder(String a, String b, int[] rank) {
        int n = Math.min(a.length(), b.length());
        for (int i = 0; i < n; i++) {
            int ra = rank[a.charAt(i) - 'a'], rb = rank[b.charAt(i) - 'a'];
            if (ra != rb) {
                return ra < rb;                    // ← first differing letter decides
            }
        }
        return a.length() <= b.length();           // ← prefix must come first
    }
}
```

Time O(n·k) | **Space** O(1)

#1055 Shortest Way to Form String

Description: Find the minimum number of subsequences of `source` whose concatenation equals `target`. Return -1 if impossible.
Intuition: Greedily consume as much of `target` as possible from each pass over `source`; if a pass makes no progress, a needed char is missing.

Variables: `count` = passes over source (subsequences) used · `i` = how much of target is consumed · `prev` = i at the start of the pass · `j` = scan index into source **Pseudocode:**

```
count = 0, i = 0
while i < length of target:
  prev = i
  scan source once; each time source[j] matches target[i], advance i
  if i did not move (i equals prev): a needed char is missing, return -1
  count += 1
return count
```

```java
class Solution {
    public int shortestWay(String source, String target) {
        int count = 0, i = 0;
        while (i < target.length()) {
            int prev = i;
            for (int j = 0; j < source.length() && i < target.length(); j++) {
                if (source.charAt(j) == target.charAt(i)) {
                    i++;                              // ← consume a target char
                }
            }
            if (i == prev) {                          // ← no progress → char missing
                return -1;
            }
            count++;
        }
        return count;
    }
}
```

Time O(n·m) | **Space** O(1)

#1108 Defanging an IP Address

Description: Defang an IP address by replacing each '.' with '[.]'.
Intuition: Append each char, expanding dots into the bracketed form.

Variables: `builder` = defanged result **Pseudocode:**

```
for each char: if it is '.', append "[.]"; else append the char
return the result
```

```java
class Solution {
    public String defangIPaddr(String address) {
        StringBuilder builder = new StringBuilder();
        for (char ch : address.toCharArray()) {
            if (ch == '.') {
                builder.append("[.]");             // ← defang dot
            } else {
                builder.append(ch);
            }
        }
        return builder.toString();
    }
}
```

Time O(n) | **Space** O(n)

#1221 Split a String in Balanced Strings

Description: Find the maximum number of balanced strings ('R' count == 'L' count) to split into.
Intuition: Track a running balance; every time it returns to zero a balanced piece has closed.

Variables: `count` = balanced pieces closed · `balance` = R count minus L count so far **Pseudocode:**

```
count = 0, balance = 0
for each char: add 1 for 'R', subtract 1 for 'L'
  whenever balance returns to 0: a balanced piece closed, count += 1
return count
```

```
class Solution {
    public int balancedStringSplit(String s) {
        int count = 0, balance = 0;
        for (char ch : s.toCharArray()) {
            balance += ch == 'R' ? 1 : -1;
            if (balance == 0) {                    // ← balanced piece closed
                count++;
            }
        }
        return count;
    }
}
```

Time O(n) | **Space** O(1)

#1328 Break a Palindrome

Description: Break a palindrome by changing one character to get the lexicographically smallest non-palindrome.
Intuition: Change the first non-'a' in the first half to 'a'; if all are 'a', change the last char to 'b'. Single-char strings can't be broken.

Variables: `n` = length · `chars` = mutable char array · `i` = scan index over the first half **Pseudocode:**

```
if length is 1: return "" (can't break)
scan the first half: at the first non-'a', change it to 'a' and return
if the whole first half is 'a': change the last char to 'b' and return
```

```
class Solution {
    public String breakPalindrome(String palindrome) {
        int n = palindrome.length();
        if (n == 1) {
            return "";                             // ← can't break a single char
        }
        char[] chars = palindrome.toCharArray();
        for (int i = 0; i < n / 2; i++) {
            if (chars[i] != 'a') {
                chars[i] = 'a';                    // ← smallest change in first half
                return new String(chars);
            }
        }
        chars[n - 1] = 'b';                        // ← all 'a': bump last char
        return new String(chars);
    }
}
```

Time O(n) | **Space** O(n)

#1392 Longest Happy Prefix

Description: Find the longest happy prefix (a non-empty prefix that is also a suffix).
Intuition: The KMP failure value at the last index is exactly the length of the longest proper prefix that is also a suffix.

Variables: `n` = length · `lps[i]` = longest prefix-also-suffix length ending at i · `j` = KMP fallback pointer **Pseudocode:**

```
build the KMP lps array over s:
  for i from 1: j = lps[i−1]; fall back while mismatch; if s[i] matches s[j], j++; lps[i] = j
return the prefix of s of length lps[last] (the longest prefix that is also a suffix)
```

```java
class Solution {
    public String longestPrefix(String s) {
        int n = s.length();
        int[] lps = new int[n];
        for (int i = 1; i < n; i++) {
            int j = lps[i - 1];
            while (j > 0 && s.charAt(i) != s.charAt(j)) {
                j = lps[j - 1];                   // ← fall back on mismatch
            }
            if (s.charAt(i) == s.charAt(j)) {
                j++;
            }
            lps[i] = j;
        }
        return s.substring(0, lps[n - 1]);        // ← longest prefix == suffix
    }
}
```

Time O(n) | **Space** O(n)

#1446 Consecutive Characters

Description: Find the maximum number of consecutive same characters in a string (the "power" of the string).
Intuition: Track the current run length, resetting on a change, and keep the maximum seen.

Variables: `max` = longest run seen · `run` = current run length **Pseudocode:**

```
max = 1, run = 1
for each index i from 1:
  if same as previous char: run += 1
  else: run = 1
  max = max(max, run)
return max
```

```java
class Solution {
    public int maxPower(String s) {
        int max = 1, run = 1;
        for (int i = 1; i < s.length(); i++) {
            if (s.charAt(i) == s.charAt(i - 1)) {
                run++;                            // ← extend run
            } else {
                run = 1;                          // ← reset
            }
            max = Math.max(max, run);
        }
        return max;
    }
}
```

Time O(n) | **Space** O(1)

#1554 Strings Differ by One Character

Description: Check if any two strings in a list differ by exactly one character (same length, same position).
Intuition: For each position, hash each word with that position wildcarded; a collision among same-length words means they differ in only that spot.

Variables: `m` = word length · `seen` = set of masked words for the current position · `j` = wildcarded position · `masked` = word with position j replaced by '*' **Pseudocode:**

```
for each position j from 0 to m−1:
  clear the seen set
  for each word: build masked = word with char j replaced by '*'
    if masked is already in seen: two words differ only at j, return true
return false
```

```
class Solution {
    public boolean differByOne(String[] dict) {
        int m = dict[0].length();
        Set<String> seen = new HashSet<>();
        for (int j = 0; j < m; j++) {
            seen.clear();
            for (String word : dict) {
                String masked = word.substring(0, j) + "*" + word.substring(j + 1);  // ← wildcard pos
                if (!seen.add(masked)) {
                    return true;
                }
            }
        }
        return false;
    }
}
```

Time O(n·m²) | **Space** O(n·m)

#1556 Thousand Separator

Description: Format an integer with a dot as the thousands separator.
Intuition: Build the result from the right, inserting a separator after every three digits.

Variables: `s` = n as a string · `builder` = result (built reversed) · `count` = digits appended since the last separator · `i` = scan index from the right **Pseudocode:**

```
s = n as a string; count = 0
for i from the last digit down to 0:
  append s[i]; count += 1
  if count is a multiple of 3 and i is not the first digit: append '.'
reverse the builder and return
```

```java
class Solution {
    public String thousandSeparator(int n) {
        String s = String.valueOf(n);
        StringBuilder builder = new StringBuilder();
        int count = 0;
        for (int i = s.length() - 1; i >= 0; i--) {
            builder.append(s.charAt(i));
            count++;
            if (count % 3 == 0 && i != 0) {        // ← separator every 3 digits
                builder.append('.');
            }
        }
        return builder.reverse().toString();
    }
}
```

Time O(d) | **Space** O(d)

#1736 Latest Time by Replacing Hidden Digits

Description: Find the latest valid time by replacing each '?' with an appropriate digit.
Intuition: Greedily choose the largest valid digit at each '?' position, respecting hour (≤ 23) and minute (≤ 59) constraints.

Variables: `chars` = mutable HH:MM array · positions 0,1 = hour digits, 3,4 = minute digits **Pseudocode:**

```
if hour tens is '?': set '2' if hour ones is '?' or <= 3, else '1'
if hour ones is '?': set '3' if hour tens is '2', else '9'
if minute tens is '?': set '5'
if minute ones is '?': set '9'
return the assembled time
```

```java
class Solution {
    public String maximumTime(String time) {
        char[] chars = time.toCharArray();
        if (chars[0] == '?') {
            chars[0] = (chars[1] == '?' || chars[1] <= '3') ? '2' : '1';  // ← max first hour digit
        }
        if (chars[1] == '?') {
            chars[1] = chars[0] == '2' ? '3' : '9';
        }
        if (chars[3] == '?') {
            chars[3] = '5';
        }
        if (chars[4] == '?') {
            chars[4] = '9';
        }
        return new String(chars);
    }
}
```

Time O(1) | **Space** O(1)

#1816 Truncate Sentence

Description: Truncate a sentence to its first `k` words.
Intuition: Copy chars until the (k)th space is reached.

Variables: `builder` = truncated result · `k` = words still allowed (decremented at each space) · `i` = scan index
Pseudocode:

```
for each char:
  if it is a space and decrementing k hits 0: stop (reached the k-th boundary)
  append the char
return the result
```

```
class Solution {
    public String truncateSentence(String s, int k) {
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == ' ' && --k == 0) {  // ← reached k-th word boundary
                break;
            }
            builder.append(s.charAt(i));
        }
        return builder.toString();
    }
}
```

Time O(n) | **Space** O(n)

#1844 Replace All Digits with Characters

Description: Replace each digit at an odd index with the letter shifted from the preceding char by that digit.
Intuition: Even indices are letters; for each odd index, shift the previous letter forward by the digit's value.

Variables: `chars` = mutable char array · `i` = odd index holding a digit · `chars[i-1]` = the preceding letter to shift
Pseudocode:

```
for each odd index i:
  replace chars[i] with the previous letter shifted forward by chars[i]'s digit value
return the array as a string
```

```
class Solution {
    public String replaceDigits(String s) {
        char[] chars = s.toCharArray();
        for (int i = 1; i < chars.length; i += 2) {
            chars[i] = (char) (chars[i - 1] + (chars[i] - '0'));  // ← shift previous char
        }
        return new String(chars);
    }
}
```

Time O(n) | **Space** O(n)